

MOVIE SCENES SCHEDULING PROBLEM

A THESIS SUBMITTED TO
THE GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES
OF
MIDDLE EAST TECHNICAL UNIVERSITY

BY

MEHTAP BARAN TEKİN

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR
THE DEGREE OF MASTER OF SCIENCE
IN
INDUSTRIAL ENGINEERING

JANUARY 2023

Approval of the thesis:

MOVIE SCENES SCHEDULING PROBLEM

submitted by **MEHTAP BARAN TEKİN** in partial fulfillment of the requirements for the degree of **Master of Science in Industrial Engineering, Middle East Technical University** by,

Prof. Dr. Halil Kalıpçılar
Dean, Graduate School of **Natural and Applied Sciences** _____

Prof. Dr. Esra Karasakal
Head of the Department, **Industrial Engineering** _____

Assoc. Prof. Dr. Sedef Meral
Supervisor, **Industrial Engineering, METU** _____

Prof. Dr. Ferda Can Çetinkaya
Co-Supervisor, **Industrial Engineering, Çankaya University** _____

Examining Committee Members:

Assist. Prof. Dr. Sakine Batun
Industrial Engineering, METU _____

Assoc. Prof. Dr. Sedef Meral
Industrial Engineering, METU _____

Prof. Dr. Ferda Can Çetinkaya
Industrial Engineering, Çankaya University _____

Assoc. Prof. Dr. Mustafa Kemal Tural
Industrial Engineering, METU _____

Assist. Prof. Dr. Diclehan Tezcaner Öztürk
Industrial Engineering, Hacettepe University _____

Date: 27.01.2023

I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all material and results that are not original to this work.

Name Last name: MEHTAP BARAN TEKİN

Signature:

ABSTRACT

MOVIE SCENES SCHEDULING PROBLEM

Baran Tekin, Mehtap
Master of Science, Industrial Engineering
Supervisor: Assoc. Prof. Dr. Sedef Meral
Co-Supervisor: Prof. Dr. Ferda Can Çetinkaya

January 2023, 108 pages

Movie scenes are not shot in the same order as they are shown in the movie. Scenes are shot independently in several distinct shooting sets with a set of actors, a film shooting crew, and several equipment as the resources. In shooting a film, several costs are incurred stemming from the usage of all these resources: wages of actors, fixed costs incurred due to hiring special shooting sets or sites, costs of using various resources like scenery, costumes and professional digital cameras, transfer costs of scenes, equipment, actors and shooting crew. Film producer companies pay the travel costs of actors to carry them to the shooting locations. However, in general, companies prefer to keep the actors in shooting sets and pay for them during their accommodation rather than covering the expensive mode of transportation. Actors are paid daily for both show-up days and do-not-show-up days while they are in shooting locations. In shooting a film, the film producer companies should consider many practical limitations such as precedence relationships among scenes, time periods indicating the earliest starting and latest completion times of scenes, and blocking time periods expressing the starting and ending of unavailability times of actors. Therefore, a movie scenes scheduling

problem (MSSP) arises from the need for finding a feasible sequence of scenes and actors considering all limitations so as to minimize the total cost of shooting.

In this study, the MSSP is defined by including the aforementioned operational limitations that have an essential effect on finding the optimal shooting schedule with the least cost possible. A mixed-integer linear programming (MILP) model is proposed, and a population-based metaheuristic, particle swarm optimization (PSO), is developed for the MSSP. Finally, the performances of the MILP model and PSO algorithm are analysed with several problem instances of different sizes and also a real script.

Keywords: Movie Scenes Scheduling, Discrete Particle Swarm Optimization Algorithm, Mixed-Integer Linear Programming

ÖZ

FİLM SAHNELERİ ÇİZELGELEME PROBLEMİ

Baran Tekin, Mehtap
Yüksek Lisans, Endüstri Mühendisliği
Tez Yöneticisi: Doç. Dr. Sedef Meral
Ortak Tez Yöneticisi: Prof. Dr. Ferda Can Çetinkaya

Ocak 2023, 108 sayfa

Film sahneleri, bir filmde gösterildikleri sırayla çekilmezler. Sahneler, kaynaklar olarak; bir grup aktör, film çekim ekibi ve çeşitli ekipman ile birkaç farklı çekim setinde birbirinden bağımsız olarak çekilir. Bir film çekilirken, tüm bu kaynakların kullanımından kaynaklanan çeşitli maliyetler oluşur: oyuncuların ödemeleri, özel çekim setlerinin veya yerlerinin kiralanması nedeniyle oluşan sabit maliyetler, sahne, kostüm, profesyonel dijital kamera gibi çeşitli kaynakların kullanım maliyetleri ve sahnelerin, ekipmanların, aktörlerin ve film çekim ekibinin transfer maliyetleri. Film yapımcı şirketleri, oyuncuları çekim yerlerine getirmek için yol masraflarını karşılarlar. Ancak, genel olarak şirketler, pahalı ulaşım yöntemini karşılamaktansa, oyuncuları çekim setlerinde tutmayı ve kaldıkları süre boyunca ücretlerini ödemeyi tercih ederler. Oyunculara hem çekim günleri hem de çekim olmayan günler için çekim yerlerinde bulundukları sürece günlük bazda ödeme yapılır. Bir film çekiminde, film yapım şirketleri, sahneler arasındaki öncelik ilişkileri, sahnelerin en erken başlama ve en geç tamamlanma zamanlarının belirlediği periyotları ve oyuncuların müsait olmadığı zamanların başlama ve bitiş

zamanlarını ifade eden engelleme zaman periyotları gibi birçok pratik sınırlamayı göz önünde bulundurulmalıdır. Bu nedenle, bir film sahnesi planlama problemi (MSSP), toplam çekim maliyetini en aza indirmek amacı ile tüm operasyonel kısıtları gözönünde bulundurarak olurlu bir sahne ve aktör sıralaması bulma gereğinden kaynaklanır.

Bu çalışmada film sahneleri çizelgeleme problemi (MSSP), minimum maliyetle uygulanabilir bir sahne sıralaması bulmada önemli bir etkiye sahip olan yukarıda bahsedilen tüm operasyonel kısıtlamaların dahil edilmesiyle tanımlanmıştır. MSSP için bir karma tam sayılı doğrusal programlama (MILP) modeli önerilmiş ve popülasyon tabanlı meta-sezgisel yöntem olan parçacık sürü optimizasyonu (PSO) yöntemi geliştirilmiştir. Son olarak, MILP modelinin ve PSO algoritmasının performansları, farklı boyutlarda birçok problem örneği ve gerçek bir film ile incelenmiştir.

Anahtar Kelimeler: Film Sahnelerini Çizelgeleme, Ayrık Parçacık Sürü Optimizasyon Algoritması, Karışık Tam Sayılı Doğrusal Programlama

To my family...

ACKNOWLEDGMENTS

I want to acknowledge and sincerely thank my supervisors, Assoc. Prof. Dr. Sedef Meral and Prof. Dr. Ferda Can Çetinkaya, for their immense motivation, knowledge, and support that carried me through all stages of my thesis.

Besides my supervisors, I thank the examining committee members for their valuable comments and suggestions.

I express my deepest gratitude to my friends who inspired me to improve my thesis study.

Last but not least, I would also like to give my warmest thanks to my caring, loving, and supportive spouse Mustafa Emrah Tekin, and my dearest father Ömer Baran for his wholehearted encouragement and patience during my thesis study.

TABLE OF CONTENTS

ABSTRACT.....	v
ÖZ	vii
ACKNOWLEDGMENTS	x
TABLE OF CONTENTS.....	xi
LIST OF TABLES.....	xiv
LIST OF FIGURES	xv
LIST OF ABBREVIATIONS.....	xvi
1 INTRODUCTION	1
2 LITERATURE REVIEW	5
2.1 Music Rehearsal Scheduling Problems	5
2.2 Talent Scheduling Problems.....	7
2.3 Movie Scenes Scheduling Problems	10
2.4 Taxonomic Review	11
3 PROBLEM DEFINITION AND FORMULATION	19
3.1 Assumptions	20
3.2 The Proposed Mathematical Model	21
3.2.1 Notation.....	21
3.2.1.1 Indices and Sets	22
3.2.1.2 Parameters.....	22
3.2.1.3 Decision Variables.....	23
3.2.2 MILP Formulation	24
4 PSO ALGORITHM	29
4.1 Brief Information about the PSO Algorithm.....	29

4.2	Related Studies on the PSO Algorithm.....	32
4.2.1	Studies Using Continuous Version of the PSO Algorithm	32
4.2.2	Studies Using Discrete Version of the PSO Algorithm.....	34
4.3	The Proposed Discrete PSO Algorithm	36
4.3.1	Phase 1: Initialization	38
4.3.2	Phase 2: PSO	38
4.3.2.1	Phase 2.1: Repairing.....	42
4.3.2.2	Phase 2.2: Improvement.....	43
5	COMPUTATIONAL EXPERIMENTS	47
5.1	Graph Types of Precedence Relationships	47
5.2	Performance Comparison of the MILP Model and PSO Algorithm	48
5.2.1	Generated Problem Instances	49
5.2.1.1	Problem Instances with 10 Scenes and 10 Actors.....	50
5.2.1.2	Problem Instances with 15 Scenes and 10 Actors.....	54
5.2.1.3	Problem Instances with 20 Scenes and 10 Actors.....	58
5.2.1.4	Problem Instances with 25 Scenes and 10 Actors.....	64
5.2.1.5	Problem Instances with 25 Scenes and 15 Actors.....	68
5.2.2	A Real Film Script Instance	72
5.3	Effects of the Numbers of Particles and Iterations on the PSO Performance.....	73
5.3.1	Evaluation over the Generated Problem Instances	73
5.3.2	Evaluation over the Real Film Script	76
5.4	Performance Evaluation of the MILP Model	81
5.5	Performance Evaluation of the PSO Algorithm	83

6	CONCLUSIONS.....	85
	REFERENCES	89
A.	FLOWCHART OF THE PROPOSED PSO ALGORITHM.....	97
A.	FLOWCHART OF THE PROPOSED PSO ALGORITHM-continued.....	98
A.	FLOWCHART OF THE PROPOSED PSO ALGORITHM- continued....	99
B.	FLOWCHART OF THE REPAIRING PROCEDURE.....	100
C.	FLOWCHART OF THE IMPROVEMENT PROCEDURE.....	101
C.	FLOWCHART OF THE IMPROVEMENT PROCEDURE- continued..	102
D.	PRECEDENCE RELATIONS AMONG THE SCENES IN THE REAL FILM SCRIPT	103
D.	PRECEDENCE RELATIONS AMONG THE SCENES IN THE REAL FILM SCRIPT- continued	104
D.	PRECEDENCE RELATIONS AMONG THE SCENES IN THE REAL FILM SCRIPT- continued	105
E.	SCENE NUMBERS IN THE PRECEDENCE DIAGRAM AND IN ŞAHİN (2007).....	106
E.	SCENE NUMBERS IN THE PRECEDENCE DIAGRAM AND IN ŞAHİN (2007)- continued.....	107
F.	SCENE-ACTOR MATRIX IN THE REAL FILM SCRIPT.....	108

LIST OF TABLES

TABLES

Table 2.1 Categories in the Taxonomic Review	12
Table 2.2 Taxonomy of the Articles (attributes 1 through 2.9.2).....	15
Table 2.3 Taxonomy of the Articles (attributes 3 through 7.2.2).....	16
Table 5.1 MILP and PSO Solutions for 10S/10A with SP-Graph	51
Table 5.2 MILP and PSO Solutions for 10S/10A with OF-Graph.....	53
Table 5.3 MILP and PSO Solutions for 15S/10A with SP-Graph	56
Table 5.4 MILP and PSO Solutions for 15S/10A with OF-Graph.....	59
Table 5.5 MILP and PSO Solutions for 20S/10A with SP-Graph	61
Table 5.6 MILP and PSO Solutions for 20S/10A with OF-Graph.....	63
Table 5.7 MILP and PSO Solutions for 25S/10A with SP-Graph	65
Table 5.8 MILP and PSO Solutions for 25S/10A with OF-Graph.....	67
Table 5.9 MILP and PSO Solutions for 25S/15A with SP-Graph	69
Table 5.10 MILP and PSO Solutions for 25S/15A with OF-Graph.....	70
Table 5.11 Effects of Numbers of Particles and Iterations – Problem Instance: 10S/10A/B-5 with OF-Graph	77
Table 5.12 Effects of Numbers of Particles and Iterations in Sequence – Problem Instance: 10S/10A/B-5 with OF-Graph.....	77
Table 5.13 Effects of Numbers of Particles and Iterations – Problem Instance: 25S/15A/T-2 with OF-Graph	78
Table 5.14 Effects of Numbers of Particles and Iterations in Sequence– Problem Instance: 25S/15A/T-2 with OF-Graph	78
Table 5.15 Effects of Numbers of Particles and Iterations – Problem Instance: Real Film Script.....	80

LIST OF FIGURES

FIGURES

Figure 2.1. Chronological Order of the Studies	17
Figure 5.1. SP-Graph for 10S/10A	50
Figure 5.2. OF-Graph for 10S/10A.....	52
Figure 5.3. SP-Graph for 15S/10A	55
Figure 5.4. OF-Graph for 15S/10A.....	57
Figure 5.5. SP-Graph for 20S/10A	60
Figure 5.6. OF-Graph for 20S/10A.....	62
Figure 5.7 SP-Graph for 25S/10A	64
Figure 5.8 OF-Graph for 25S/10A.....	66
Figure 5.9 Quality of Solutions based on Percent Deviation (PD)	74
Figure 5.10 Effects of Numbers of Particles and Iterations on Computational Time	79
Figure 5.11 Effects of Numbers of Particles and Iterations – Problem Instance: Real Film Script	82

LIST OF ABBREVIATIONS

ACO	Ant Colony Optimization
CP	Constraint Programming
DP	Dynamic Programming
GA	Genetic Algorithm
LPV	Largest Position Value
MILP	Mixed-Integer Linear Programming
MRSP	Music Rehearsal Scheduling Problem
MSSP	Movie Scenes Scheduling Problem
PSO	Particle Swarm Optimization
TS	Tabu Search
TSP	Talent Scheduling Problem

CHAPTER 1

INTRODUCTION

A movie consists of many distinct scenes where a set of actors participate. The scenes in a movie are not generally shot in the same sequence in which they appear in the final version. After scenes are shot independently, they are edited to create a movie. While shooting a scene, many actors are required in remote filming locations to perform their roles. The film production company also pays for the travel cost of actors from their residences to the filming sites. In general, the travel cost of actors turns out to be high due to the selection of expensive modes of transportation like business class commercial airlines or private jets. In addition, the movement of actors back and forth between their residence and filming sites, when needed, is not always sustainable and practical during a movie scenes shooting. This may cause burnout syndrome in actors and an increase in total filming costs. As a result, most film producer companies prefer to keep actors at the shooting sites from the first day until the last day they are needed. Hence, actors are paid daily for both the days when they show up and do not show up. The days during which an actor is present in the shooting location but not required for performance are called “hold days” for the actor.

Therefore, a MSSP arises for shooting scenes at the predetermined shooting sites with the actors having roles in the scenes. This problem is addressed to find a feasible sequence of scenes so as to minimize the total cost of a movie shoot which is initially introduced as a *music rehearsal scheduling problem* (MRSP) by Adelson et al. (1976) to obtain a sequence of music pieces rehearsed with different sets of players. The MRSP aims to minimize the total rehearsal cost arising from the equal wages of players on their show-up days. A different version of a movie scene scheduling problem called a *talent scheduling problem* (TSP) is developed

by Cheng et al. (1993). In the problem they introduce, each scene is a shooting day, meaning that the duration of scenes is assumed to be identical, whereas the daily wage of actors is different. The objective is to find a sequence of scenes to minimize the total cost of talent.

MSSP is firstly constructed as an integer linear programming model to minimize the transfer cost of scenes and wages of actors by Liu et al. (2019). However, in a realistic movie shooting, plenty of factors should be considered to determine a feasible sequence of scenes: logistic factors, precedence relationships among scenes, time windows of scenes, unavailability time periods of actors, and several cost elements. The corresponding total cost is composed of the wages of actors, fixed cost incurred due to hiring special shooting sets or sites, cost of using various resources like scenery, costumes and professional digital cameras, and transfer cost of scenes, equipment, actors and shooting crew. Logistical factors are also related to the cost factors. For instance, hiring a shooting set closer to the next shooting place affects the transfer cost of the scenes, equipment, actors, and shooting crew. Precedence relations among scenes indicate the limitations for the shooting order (sequence) of scenes. The time window of a scene indicates the earliest starting and latest completion times of the scene. Actors may go on-leave at some certain times of the shooting horizon due to personal affairs like vacation, health, family issues, etc. In addition, an actor may have to be at a different movie shooting set at the same time. These situations are stated as unavailability or blocking time periods of actors.

To the best of our knowledge, no such study exists in the literature that takes into account all of these real operational limits in scheduling movie scenes. Hence, we define the real MSSP, and provide both an exact method and a heuristic solution method that provide optimal or near-optimal solutions.

We propose a taxonomy for the relevant literature by first categorizing the problems as music rehearsal scheduling, talent scheduling, and movie scenes

scheduling; and then examine the studies with respect to their several characteristics.

In addressing the MSSP in this study, we propose an MILP model for this problem. The objective of the proposed model is to minimize the total cost of a movie's shooting. The objective function then consists of actors' wages during their available time, transfer costs of scenes, equipment, actors, and shooting crew, fixed cost incurred due to hiring special shooting sets or sites, and cost of utilizing various resources like scenery, costumes, and professional digital cameras. The MILP formulation of the problem includes constraints related to the satisfaction of precedence relationships among scenes, earliest starting and latest completion times of scenes complying with the time windows of scenes, and blocking time periods of actors during which they go on-leave for a prespecified duration. In addition, at most, one scene can be shot at a time because of a single shooting crew. In light of this information, one of the metaheuristic optimization algorithms, particle swarm optimization (PSO), is devised to solve especially the large-scale (real-life sized) MSSP's for which the MILP model via the CPLEX solver cannot provide even feasible solutions in reasonable computational times. The performances of both methods are evaluated based on several problem instances generated, including an instance of a real film script, as well.

The remainder of this thesis is organized as follows. In Chapter 2, the relevant studies on the scheduling of artistic activities in literature are reviewed, a taxonomy is proposed for these problem environments based on their attributes, and chronological order of them is provided to see how the studies are improved over time. In Chapter 3, the MSSP environment and its characteristics are defined. The MILP model is proposed for the MSSP as defined in the thesis. In Chapter 4, first, the background information on the PSO algorithm is introduced, and its variants in the literature are reviewed; then, the PSO algorithm proposed for the problem is explained in detail. In Chapter 5, computational experiments assessing the performance of the proposed solution methods are presented. Chapter 6 concludes

the thesis with its contributions to the literature and points out future research directions.

CHAPTER 2

LITERATURE REVIEW

A movie scene scheduling problem is similar to both music rehearsal scheduling and talent scheduling problems. Therefore, in this chapter, a substantial amount of studies about movie scenes scheduling problems and similar ones are reviewed comprehensively and classified in three groups:

- (i) Studies about music rehearsal scheduling problems in Section 2.1.
- (ii) Studies about talent scheduling problems in Section 2.2.
- (iii) Studies about movie scenes scheduling problems in Section 2.3.

In Section 2.4, a taxonomy is proposed according to the main attributes considered for these three types of problems. In Section 2.5, the chronological orders of the relevant studies are provided.

2.1 Music Rehearsal Scheduling Problems

A music rehearsal problem is closely related to a movie scene scheduling problem. This problem is studied to find a sequence of music pieces rehearsed with different player sets that aims to minimize the total rehearsal cost arising from several rehearsals and show-up days experienced by all players.

A simplified/restricted version of TSP is first defined by Adelson et al. (1976) as a music rehearsal scheduling problem where the costs of all players are uniform. They develop a dynamic programming (DP) model, whose objective is to find an arrangement of music pieces so as to minimize the total required man-hours of all players.

Smith (2003, 2005) presents the basic principles of constraint programming (CP) to find a sequence in a music rehearsal scheduling problem where music pieces have different lengths. Like scenes in talent scheduling problems, the music pieces are sequenced to minimize the players' total waiting (idling) times in one day. This model is also utilized in talent scheduling problems with some improvements. Gregory et al. (2004) develop planning and model-checking techniques to solve the music rehearsal scheduling problem to speed up the computations. They compare their two methods with the CP proposed by Smith (2003). They prove that the planning technique outperforms both CP and model-checking methods regarding computational efficiency and solution quality. All works related to music rehearsal scheduling problems assume that music pieces are sequenced continuously in a single day from the first to the last one.

On the other hand, Sakulsom and Tharmmaphornphilas (2011) and Sakulsom and Tharmmaphornphilas (2014) define their problems as the music pieces are allowed to be rehearsed over multiple days. Sakulsom and Tharmmaphornphilas (2011) consider the case where music pieces have the same duration, whereas Sakulsom and Tharmmaphornphilas (2014) assume that music pieces have different durations. Since the daily wages of all players are unique in both problems stated by Sakulsom and Tharmmaphornphilas (2011), and Sakulsom and Tharmmaphornphilas (2014), the objective is to minimize the number of days required for each player together with the resulting waiting times. Sakulsom and Tharmmaphornphilas (2014) also limit the daily operating hours. They propose a two-phase approach; first, the concept of cell formation for initial solutions and then the integer programming model for sequences of music pieces that minimize total waiting times. This two-phase approach may cause sticking at the local optimum. Nonetheless, Jarumaneeroj and Sakulsom (2021) provide a single-stage integer programming model such that setup times are considered according to the change in the player sets between two consecutive music pieces. They also construct an integer programming model without setup times, similar to the

multiple-day music rehearsal problem in Sakulsom and Tharmmaphornphilas (2014).

Pakprajum and Jarumaneeroj (2017) present an integrated MILP to solve a multiple-day music rehearsal problem in one stage. They develop a model so that the total cost of rehearsing, including rental cost and wage of players, is minimized. Liu (2016) proposes a mixed integer programming model and a genetic algorithm (GA) to minimize the total cost for players' waiting time by finding the optimum sequence of the music pieces with unequal durations. Performance time is formulated as the sum of certain and uncertain adjustment times. It is assumed that uncertain adjustment time follows the normal distribution. On the other hand, Ding et al. (2018) state that a large amount of data should be collected to say that the data follow the normal distribution. Therefore, they use the robust optimization method with uncertain durations, expressed as a continuous bounded interval with lower and upper bounds, to eliminate this assumption.

Hill and Velednitsky (2021) study the music rehearsal scheduling problem as a multi-objective optimization problem with three objectives. They use these objective functions hierarchically and solve the problem using three models.

2.2 Talent Scheduling Problems

A talent scheduling problem is also closely related to a movie scene scheduling problem. This problem deals with finding a sequence of scenes for a set of actors to minimize the total cost. Total cost is affected by the wages of all actors according to their participation from the first scene to the last scene.

Cheng et al. (1993) introduce a talent scheduling problem where each scene is a shooting day, meaning that the durations of all scenes are assumed to be identical. The daily wages of actors are different. They aim to sequence a series of shooting scenes to minimize the total cost of talent. Talent cost occurs due to talents' presence at the shooting place. They also prove that the problem is strongly NP-

hard even when each actor appears in only two scenes, and daily wages of actors and all scene durations are identical. They present a branch and bound method and a two-phase heuristic algorithm to solve the problem. They state that the proposed branch and bound method can solve the problem for only small dimensions, up to 14 days and 14 talents. For problems larger than 14 days and 14 talents, they report that their proposed heuristic method performs successfully in a reasonable computational time.

Nordström and Tufekci (1994) propose the first metaheuristic approach, several hybrid genetic algorithms, using a limited pairwise interchange method within the simple genetic algorithm to solve the problem presented by Cheng et al. (1993). This algorithm outperforms the two-phase heuristic developed by Cheng et al. both in the quality of the solution and in computational times. Fink and Voß (1999) apply a simulated annealing and several tabu search (TS) algorithms to solve the same problem. Kochetov (2011) develops three new heuristic algorithms, including a hybrid simulated annealing algorithm, a stochastic TS algorithm, and a genetic local search to solve the same problem. They claim that the algorithms can find the optimal solutions for randomly generated problem instances up to 100 days and 100 talents within a few minutes.

Garcia de la Banda et al. (2011) apply an efficient dynamic programming (DP) formulation to solve the problem proposed by Cheng et al. (1993). Differently, they introduce the problem where scenes may have different durations and actors may have non-identical daily wages. They utilize call-based best-first dynamic programming formulation, which is also efficient for large-scale problems using preprocessing and restricting the search. Their solutions indicate that dynamic programming outperforms the two-phase heuristic proposed by Cheng et al. (1993) in terms of both solution quality and computational time. Cheng et al. (2017) contribute a systematic enhancement of lower bounds for the talent scheduling problem defined by Cheng et al. (1993). They compare the lower bounds found by Cheng et al. (1993) and their tightened lower bounds using the maximum-weighted matching approach and the maximum-weighted 3-grouping approach.

Qin et al. (2014, 2016) introduce an integer programming model and an enhanced branch and bound methods for the problem proposed by Garcia de la Banda et al. (2011). However, the integer programming model developed can solve the problem optimally for only small instances, up to 10 scenes and 5 actors. In addition, an enhanced branch and bound method is improved by well-designed dominance checkers and state caching strategies which outperforms the already developed algorithms. Zhang et al. (2019) present an exact application of the developed enhanced branch and bound method by Qin et al. (2014, 2016). They also compare the results of CP by Smith (2003, 2005), DP by Garcia de la Banda et al. (2011), and the proposed enhanced branch and bound method by Qin et al. (2014, 2016). They prove that the enhanced branch and bound method requires less computational time since it produces fewer nodes than CP and DP.

Since any previously developed algorithms are stuck in dimension disaster, Wang (2015) aims to solve the talent scheduling problem for large-scale instances in such a way that minimizes total talent costs by GA with an elitist strategy and a discrete PSO algorithm. Their PSO algorithm gives better results than their GA for large-scale problems. Wang et al. (2016) generalize the talent scheduling problem introduced by Cheng et al. (1993). They include that more than one scene can be scheduled in a single shooting day as they do not exceed the allowable daily operating capacity. However, while trying to decrease the number of shooting days, actors' holding costs may increase and vice versa. Therefore, integer linear programming is developed to balance the required shooting days and actors' holding costs by minimizing the sum of the actors' holding costs and the operating cost of shooting days. Since the problem is NP-hard, some meta-heuristic methods are utilized to find near-optimum solutions.

Ranjbar and Kazemi (2018) introduce a generalized variable neighborhood search (GVNS) algorithm to solve the same problem in Garcia de la Banda et al. (2011). The aim is to minimize the total holding cost of the actors. Holding cost occurs when an actor is not playing in a scene but just waiting until s/he appears in her/his next scene.

2.3 Movie Scenes Scheduling Problems

A movie scene scheduling problem involves sequencing the scenes to minimize the total cost of shooting a movie.

Bomsdorf and Derigs (2008) study a movie scenes scheduling problem, a generalized version of the talent scheduling problem introduced by Garcia de la Banda et al. (2011). A conceptual model is presented as a decision support tool by considering many realistic constraints such as unavailable working times of the actors, precedence relations among scenes, required resources, and permitted time windows for the scenes. Liu et al. (2019) construct an integer linear programming for a movie scenes scheduling problem that is affected by many realistic factors. Upon this, meta-heuristic algorithms such as the TS and PSO algorithms are developed for large-scale problem instances. For large-scale problems, it is proven that the TS algorithm gives better results than PSO algorithm provides, whereas, for small-scale problems, both algorithms are applicable. They consider the transfer costs of scenes from one shooting set to another.

Moreover, actors' daily wages and scenes' shooting durations can be non-identical. The objective is then to minimize the total cost of movie shooting consisting of the daily salary of actors and the transfer cost of scenes. Similarly, Long and Zhao (2020) establish an integer linear programming formulation for the same problem in Liu et al. (2019) by considering the transfer cost of scenes regarding the locations, different daily wages of actors and shooting durations of scenes. In addition, they utilize TS, PSO, and ACO algorithms to study a movie scenes scheduling problems for medium-scale problem instances. They indicate that the results of the ACO algorithm are the best among them, and the results of the TS algorithm are better than those of the PSO. In terms of computational time, the TS algorithm runs faster than the others, and running of the ACO algorithm may be faster than that of the PSO according to the number of ants.

2.4 Taxonomic Review

Taxonomy is a systematic literature review tool for the storage of knowledge efficiently. It gives a panoramic view of the entire study on a selected topic. It describes and classifies the studies according to the predefined attributes/dimensions for a specific problem domain. It also allows the identification of the possible combinations of these attributes and boundaries of the problem domain.

Our taxonomy is created in an arborescent way, that is, branching levels from top to bottom are used. The categories and sub-categories are presented in Table 2.1.

The first level categories of branching are composed of:

- i. Type of Problem
- ii. Scene Characteristics
- iii. Actor Characteristics
- iv. Working Characteristics
- v. Crew Characteristics
- vi. Equipment Characteristics
- vii. Shooting Characteristics

Studies are classified within the first attribute set based on their content and problem descriptions. Studies are classified whether they are related to a music rehearsal, talent, or movie scenes scheduling problem. After identifying the type of the problem for those studies, the rest of the characteristics are introduced.

In the second attribute set, scene characteristics are subdivided into nine sub-characteristics. To begin with, the duration of the scenes may be equal or not. Moreover, there may be precedence relations between scenes.

Table 2.1 Categories in the Taxonomic Review

1.	Type of Problem	3.5.	Absenteeism
1.1.	Music Rehearsal Scheduling	3.5.1.	Yes
1.2.	Talent Scheduling	3.5.2.	No
1.3.	Movie Scenes Scheduling	3.6.	Stuntman/Optional
		3.6.1.	Yes
2.	Scene Characteristics	3.6.2.	No
2.1.	Duration		
2.1.1.	Equal	4.	Working Characteristics
2.1.2.	Different	4.1.	Number of Scenes per Day
2.2.	Precedence	4.1.1.	Single
2.2.1.	Yes	4.1.2.	Multiple
2.2.2.	No	4.2.	Limited Operating Hours in a Day
2.3.	Transfer Cost	4.2.1.	Yes
2.3.1.	Yes	4.2.2.	No
2.3.2.	No	4.3.	Maximum Number of Consecutive Workdays
2.4.	Rental Cost	4.3.1.	Yes
2.4.1.	Yes	4.3.2.	No
2.4.2.	No		
2.5.	Time Window	5.	Crew Characteristics
2.5.1.	Yes	5.1.	Size
2.5.2.	No	5.1.1.	Single
2.6.	Setup Cost	5.1.2.	Multiple
2.6.1.	Yes	5.2.	Blocking Time
2.6.2.	No	5.2.1.	Yes
2.7.	Atmosphere	5.2.2.	No
2.7.1.	Yes	5.3.	Crew Operating Cost
2.7.2.	No	5.3.1.	Yes
2.8.	Break Between Scenes	5.3.2.	No
2.8.1.	Yes		
2.8.2.	No	6.	Equipment Characteristics
2.9.	Chronological Order of Scenes	6.1.	Standart Equipment Capacity
2.9.1.	Yes	6.1.1.	Unlimited
2.9.2.	No	6.1.2.	Limited
		6.2.	Special Equipment Required
3.	Actor Characteristics	6.2.1.	Yes
3.1.	Wage	6.2.2.	No
3.1.1.	Equal		
3.1.2.	Different	7.	Shooting Characteristics
3.2.	Payment Type	7.1.	Sequence
3.2.1.	Continuous	7.1.1.	Sequential
3.2.2.	Interrupted	7.1.2.	Parallel
3.3.	Sending Home Cost	7.2.	Location
3.3.1.	Yes	7.2.1.	Single
3.3.2.	No	7.2.2.	Multiple
3.4.	Blocking Time		
3.4.1.	Yes		
3.4.2.	No		

Time and atmosphere limitations may be imposed in shooting a scene; a scene may be required to be shot within a specified time interval. For instance, winter conditions with snow and sunlight may be required, whereas moonlight and summer nights may be required for some scenes to be shot. Some scenes may need to be shot interior or exterior. Moreover, setup, transfer, and rental costs for shooting scenes may be included or ignored. The film producer may give breaks between scenes so that actors and shooting crew can rest.

Furthermore, shooting scenes in a chronological order may be preferred or not by the film producer. Shooting scenes in a chronological order may increase the concentration of actors. As a result of this, the duration of a movie shooting may become shorter. All of these options affect the total cost of movie shooting.

The third category is composed of actor characteristics with six sub-characteristics. The wages of all actors may be equal or different. In general, actors are paid from the first day until the end of the last scene they appear at the shooting site, even though their performances are not needed in all of the scenes in this time interval. However, the film producer company may decide on the payment type as either continuous or interrupted. In other words, companies may prefer to avoid paying extra money for days when actors are not required to participate. In addition, some actors may want to go on leave at certain times during the film shooting. This type of leave for a specified time interval is called blocking time for actors. While the absence of some actors may have a stopping effect in some movie shootings, this situation may be ignored in some movie shootings. Using stunts also increases the cost of a movie shooting because an additional actor is paid for each day in which the real one participates.

The fourth category is about the working characteristics with three sub-classes. In a single day, the film producer may prefer only one scene shooting. However, in some cases, more than one scene may be shot in a single day. The number of working hours in a day may affect the actors' and shooting crew's health and also the quality of the movie. Therefore, the maximum number of working hours in a

day may be predefined so that actors and the shooting crew have time to rest. Similarly, the film producer may limit the maximum number of consecutive working days in order to give a break for both actors and shooting crew.

The fifth category is organized with three sub-categories about crew characteristics. There may be more than one shooting crew that simultaneously allows parallel shootings at different sites. Like the actors, people in the crew may also have a blocking time period during movie shooting time. Moreover, crew operating costs may be incurred or ignored during movie shooting.

The sixth category is composed of equipment characteristics with two sub-classes. The capacity of equipment used in a movie shooting may be limited or unlimited. After a while, renewal of equipment may be required during a long movie shooting. However, the capacity of equipment may be assumed as unlimited and can be used during the whole movie shooting time. Special cameras, lighting equipment, and effects may be needed in some films, but this exclusive equipment may not be required in all films.

In the last attribute set, shooting characteristics are subdivided into two sub-characteristics. This attribute is also related to the size of the shooting crews. Parallel shootings can be organized if there is more than one shooting crew; but, if there is only one shooting crew, scenes can only be shot sequentially. In other words, only one scene can be shot at a time in the case of one shooting crew. Similarly, more than one shooting location may be hired and organized to shoot scenes in parallel in the case of more than one shooting crew.

The aforementioned attributes are firstly adapted from the study about formulating a conceptual model for the movie scenes scheduling problem introduced by Bomsdorf and Derigs (2008). However, they are extended in the light of the book about complete film production by Honthaner (2013). Tables 2.2 and 2.3 provide the taxonomy for the reviewed papers in the literature. It can be concluded from the taxonomy that there is a gap in the literature on the scope of movie scenes scheduling problems dealing with all realistic factors. Therefore, our study in this

thesis can be leading for future studies that have similar characteristics to the movie scenes scheduling problem. Secondly, the studies (with their numbers) on each of the three types of problems are organized in the chronologically ordered graphs in Figure 2.1.

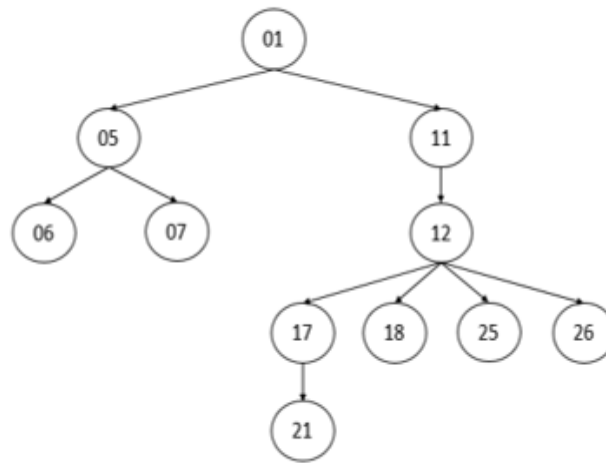
Table 2.2 Taxonomy of the Articles (attributes 1 through 2.9.2)

	Article	1.	1.1.	1.2.	1.3.	2.	2.1.	2.1.1.	2.1.2.	2.2.	2.2.1.	2.2.2.	2.3.	2.3.1.	2.3.2.	2.4.	2.4.1.	2.4.2.	2.5.	2.5.1.	2.5.2.	2.6.	2.6.1.	2.6.2.	2.7.	2.7.1.	2.7.2.	2.8.	2.8.1.	2.8.2.	2.9.	2.9.1.	2.9.2.
1	Adelson et al. (1976)	✓						✓						✓			✓			✓			✓			✓			✓			✓	
2	Cheng et al. (1993)		✓					✓				✓		✓			✓			✓			✓			✓			✓			✓	
3	Nordström & Tufekci (1994)		✓					✓				✓		✓			✓			✓			✓			✓			✓			✓	
4	Fink & Voß (1999)		✓					✓						✓			✓						✓			✓							✓
5	Smith (2003)		✓						✓			✓		✓			✓					✓			✓							✓	
6	Gregory et al. (2004)		✓						✓			✓		✓			✓					✓			✓							✓	
7	Smith (2005)		✓						✓			✓		✓			✓					✓			✓							✓	
8	Bomsdorf & Derigs (2008)			✓				✓		✓				✓			✓			✓						✓				✓			✓
9	Garcia de la Banda et al. (2011)		✓						✓			✓		✓			✓					✓			✓					✓			✓
10	Kochetov (2011)		✓					✓				✓		✓			✓					✓			✓					✓			✓
11	Sakulsom & Tharmmaphornphilas (2011)	✓						✓				✓		✓			✓					✓			✓					✓			✓
12	Sakulsom & Tharmmaphornphilas (2014)	✓							✓			✓		✓			✓					✓			✓					✓			✓
13	Qin et al. (2014)		✓						✓					✓			✓					✓			✓					✓			✓
14	Wang (2015)		✓						✓			✓		✓			✓					✓			✓					✓			✓
15	Qin et al. (2016)		✓						✓			✓		✓			✓					✓			✓					✓			✓
16	Wang et al. (2016)		✓						✓			✓		✓			✓					✓			✓					✓			✓
17	Liu (2016)		✓						✓			✓		✓			✓					✓			✓					✓			✓
18	Pakprajum & Jarumaneeroj (2017)		✓						✓			✓		✓		✓			✓			✓			✓					✓			✓
19	Cheng et al. (2017)		✓					✓				✓		✓			✓					✓			✓					✓			✓
20	Ranjbar & Kazemi (2018)		✓						✓			✓		✓			✓					✓			✓					✓			✓
21	Ding et al. (2018)		✓						✓			✓		✓			✓					✓			✓					✓			✓
22	Zhang et al. (2019)		✓						✓			✓		✓			✓					✓			✓					✓			✓
23	Liu et al. (2019)			✓				✓				✓		✓			✓					✓			✓					✓			✓
24	Long & Zhao (2020)			✓				✓				✓		✓			✓					✓			✓					✓			✓
25	Jarumaneeroj & Sakulsom (2021)		✓						✓			✓		✓			✓					✓	✓		✓					✓			✓
26	Hill & Velednitsky (2021)		✓						✓			✓		✓			✓					✓			✓					✓			✓

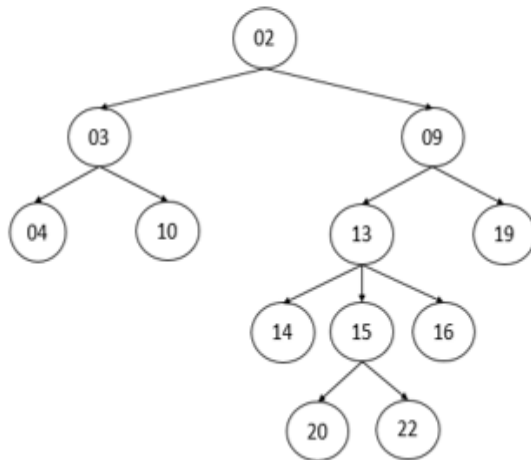
Table 2.3 Taxonomy of the Articles (attributes 3 through 7.2.2)

[illegible]

Music Rehearsal Scheduling Problems



Talent Scheduling Problems



Movie Scenes Scheduling Problems

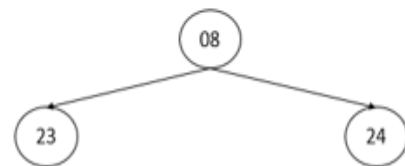


Figure 2.1. Chronological Order of the Studies

CHAPTER 3

PROBLEM DEFINITION AND FORMULATION

Movie scenes are not shot in the same order as they are shown in the movie. Therefore, our problem is mainly related to the movie scenes scheduling problem (MSSP). This scheduling problem deals with finding a feasible shooting sequence of scenes and actors so that the total cost incurred in shooting a film is minimized. The corresponding total cost is composed of the wages of actors, fixed costs due to hiring special shooting sets or sites, cost of using various resources like scenery, costumes and professional digital cameras, and transfer cost of scenes, equipment, actors and shooting crew. Generally, most studies consider only the cost of actors, including the daily wages of actors, and transfer costs of scenes. However, our problem considers all realistic factors affecting the total cost of a film's shooting. We construct an MILP model with some extensions in light of the realistic limitations and cost factors in shooting. These extensions are stated below:

- Precedence relationships among the scenes
- Blocking time periods of actors
- Time windows of scenes
- Resource usage costs and their fixed costs incurred during a movie shooting

Precedence relations among scenes are to be obeyed in ordering the shooting of scenes. Based on the precedence relations, some scenes must be shot before their successor scenes; hence, all predecessor scenes of a scene must have been shot before that scene can be started. On the other hand, blocking time period for an actor refers to the unavailable time period of the actor during the shooting horizon. During a specific time interval, actors may not be available due to personal affairs like vacation, health issues, family issues, etc; in addition, they may have to be at another movie shooting site during the same shooting horizon. Therefore, the

scenes of an actor should not be scheduled in her/his unavailable time periods, called as the blocking time periods. As for the scenes' shooting time limitations, a time window is expressed with an earliest starting time and a latest completion time for shooting a scene. It is then defined as the time interval during which a scene can be shot. For example, a time window may correspond to the temporal limitations such as day or night, sunset, sunrise, winter/summer, etc., when the scenes are scheduled.

Resource usage costs are incurred from the first time of use to the last scene's completion where the resource is needed for the last time. In a movie shooting, special resources like scenery, costumes, and professional digital cameras can be utilized to shoot a scene. Fixed costs are incurred during a movie shooting due to hiring special shooting sets as well as renting a house. So the total cost mainly consists of four parts: the wages of all actors, transfer costs of scenes, equipment, actors and shooting crew, resource usage cost, and their fixed costs incurred during a film shooting.

3.1 Assumptions

In modelling the MSSP under study, we make the following assumptions:

- The scenes composing a movie have deterministic known durations.
- Shooting of all scenes composing a movie should be completed by a certain due date.
- Only one scene can be shot at a time since there is a single shooting crew.
- Shooting of scenes can be performed at different sites; hence, actors, shooting crew and all resources can be transferred from one site to another.
- There are precedence relations among the scenes, that is, a scene can be started upon completion of its immediate predecessors.
- All predecessor scenes must be shot before a successor scene can start.
- Pre-emption of scenes is not allowed, that is, once a scene is started to be shot, it cannot be interrupted until it is completed.

- Two dummy scenes are defined: one is for the beginning, and the other is for the ending of the sequence. The processing times of the dummy scenes are zero.
- Transfer costs among scenes are sequence-dependent, and includes both setup costs of the actors and movement cost of equipment and crew.
- Special resources (scenery, costumes and different kinds of professional digital cameras) are renewable.
- The resources are utilized for a specific set of scenes.
- Resources usage costs are incurred from the first to the last scene in which they are needed.
- Some actors may not be available at all times during shooting of all scenes, so their unavailable (blocking) time periods are to be obeyed; actors are not paid during their blocking time period.
- During the shooting all movie scenes, actors are allowed to have only one period of unavailable time.
- The time windows of scenes are hard time windows.

3.2 The Proposed Mathematical Model

In this section, a mixed integer linear programming (MILP) model is developed for the MSSP. The notation used in the model and the model formulation are presented in Sections 3.2.1 and 3.2.2, respectively.

3.2.1 Notation

The indices and sets, parameters, and decision variables are described in Sections 3.2.1.1, 3.2.1.2, and 3.2.1.3, respectively.

3.2.1.1 Indices and Sets

The indices and sets utilized in the mathematical model are described below:

i, j	Indices of scenes
k	Index of actors
h	Index of resources
A	Set of all actors: $k \in A$
AU	Set of actors that have an unavailable time period
R	Set of all resources: $h \in R$
S	Set of all real scenes: $i, j \in S$
S^0	Set of all real scenes and dummy scene 0 where the crew begins shooting
S^T	Set of all real scenes and dummy scene T where the crew finishes shooting
$\bar{S}$	Set of all real scenes and the two dummy scenes, 0 and T
SP	Set of scenes that have one or more predecessors
IP_j	Set of immediate predecessors of scene j
SW	Set of scenes that have a time window
SU_k	Set of scenes in which actor k with an unavailable time plays
SM_k	Set of scenes in which actor k plays
SR_h	Set of scenes that require resource h

3.2.1.2 Parameters

The parameters used in the MILP model are listed below:

CM_{ij}	Cost of moving from scene i site to scene j site
RC_h	Daily usage cost of resource h
FC	Daily fixed cost of shooting
W_k	Daily wage of actor k

D_i	Number of days required to shoot scene i
ES_i	Earliest starting time for shooting scene i
LF_i	Latest completion time for shooting scene i
BU_k	Beginning of unavailable time of actor k
EU_k	Ending of unavailable time of actor k
P_{ik}	Set to 1 if actor k plays in scene i , and 0 otherwise
DD	Due date for completing all scenes shooting

3.2.1.3 Decision Variables

The decision variables of the model are provided below:

$$x_{ij} = \begin{cases} 1, & \text{if scene } j \in \bar{S} \text{ immediately follows scene } i \in \bar{S} \\ 0, & \text{else} \end{cases}$$

$$y_{ik} = \begin{cases} 1, & \text{if scene } i \in \bar{S} \text{ is shot after the ending of unavailable time of} \\ & \text{actor } k \in A \text{ playing in scene } i \in \bar{S} \\ 0, & \text{else} \end{cases}$$

$$pr_k = \begin{cases} 1, & \text{if payment reduction occurs for actor } k \in AU \\ 0, & \text{else} \end{cases}$$

c_i	Completion time of scene $i \in \bar{S}$
est_h	Starting time of the scene that resource $h \in R$ is used for the first time
lct_h	Completion time of the scene that resource $h \in R$ is used for the last time
$minst_k$	Starting time of the first scene in which actor $k \in A$ plays
cls_k	Completion time of the last scene in which actor $k \in A$ plays
sfs_{ik}	Starting time of scene $i \in \bar{S}$ in which actor $k \in A$ plays

$minstr$	Starting time of the first scene
$maxcomp$	Completion time of the last scene
$duration$	Duration of the movie shooting

3.2.2 MILP Formulation

Using the sets, parameters, and decision variables as defined above, the MILP model for the MSSP is presented as follows:

$$\begin{aligned}
\text{Minimize } & \sum_{k \in A} W_k \cdot (cls_k - minstk_k) - \sum_{k \in AU} W_k \cdot pr_k \cdot (EU_k - BU_k) + \sum_{i \in S} \sum_{j \in S} CM_{ij} \cdot x_{ij} \\
& + \sum_{h \in R} RC_h \cdot (lct_h - est_h) + FC \times duration
\end{aligned} \tag{3.1}$$

Subject to

$$\sum_{j \in S^T} x_{0j} = 1 \tag{3.2}$$

$$\sum_{i \in S^0} x_{iT} = 1 \tag{3.3}$$

$$\sum_{\substack{j \in S^T \\ i \neq j}} x_{ij} = 1 \quad \forall i \in S \tag{3.4}$$

$$\sum_{\substack{j \in S^0 \\ i \neq j}} x_{ji} = 1 \quad \forall i \in S \tag{3.5}$$

$$x_{ij} + x_{ji} \leq 1 \quad \forall i, j \in \bar{S}, i \neq j \tag{3.6}$$

$$c_j - c_i - D_j + M(1 - x_{ij}) \geq 0 \quad \forall i \in S^0, \forall j \in S^T, i \neq j \quad (3.7)$$

$$c_j - D_j \geq c_i \quad \forall j \in SP, \forall i \in IP_j \quad (3.8)$$

$$ES_i \leq c_i - D_i \quad \forall i \in SW \quad (3.9)$$

$$c_i \leq LF_i \quad \forall i \in SW \quad (3.10)$$

$$c_i - BU_k \leq M(y_{ik}) \quad \forall k \in AU, \forall i \in SU_k \quad (3.11)$$

$$EU_k + D_i - c_i \leq M(1 - y_{ik}) \quad \forall k \in AU, \forall i \in SU_k \quad (3.12)$$

$$lct_h \geq c_i \quad \forall i \in SR_h \quad (3.13)$$

$$est_h \leq c_i - D_i \quad \forall i \in SR_h \quad (3.14)$$

$$pr_k \leq \sum_{i \in SU_k} y_{ik} \quad \forall k \in AU \quad (3.15)$$

$$pr_k \leq \sum_{i \in SU_k} P_{ik} - \sum_{i \in SU_k} y_{ik} \quad \forall k \in AU \quad (3.16)$$

$$sfs_{ik} = P_{ik}(c_i - D_i) \quad \forall i \in SM_k, \forall k \in A \quad (3.17)$$

$$cls_k \geq P_{ik}(c_i) \quad \forall i \in SM_k, \forall k \in A \quad (3.18)$$

$$minst_k \leq sfs_{ik} \quad \forall i \in SM_k, \forall k \in A \quad (3.19)$$

$$minstr \leq sfs_{ik} \quad \forall i \in SM_k, \forall k \in A \quad (3.20)$$

$$maxcomp \geq c_i \quad \forall i \in S^0 \quad (3.21)$$

$$duration = maxcomp - minstr \quad (3.22)$$

$$c_i \leq DD \quad \forall i \in \bar{S} \quad (3.23)$$

$$lct_h, est_h \geq 0 \quad \forall h \in R \quad (3.24)$$

$$cls_k, minst_k \geq 0 \quad \forall k \in A \quad (3.25)$$

$$c_i \geq 0 \quad \forall i \in \bar{S} \quad (3.26)$$

$$sfs_{ik} \geq 0 \quad \forall i \in SM_k, \forall k \in A \quad (3.27)$$

$$sfs_{ik} \geq 0 \quad \forall i \in SM_k, \forall k \in A \quad (3.27)$$

$$duration, maxcomp, minstr \geq 0 \quad (3.28)$$

$$x_{ij} \in \{0, 1\} \quad \forall i, j \in \bar{S} \quad (3.29)$$

$$y_{ik} \in \{0, 1\} \quad \forall i \in \bar{S}, \forall k \in A \quad (3.30)$$

$$pr_k \in \{0, 1\} \quad \forall k \in A \quad (3.31)$$

The objective function (3.1) minimizes the total cost of shooting movie scenes. This total cost includes the wages of actors from the first day until the last day they are needed to shoot the corresponding scenes with the exception of unavailable times of actors, the transfer costs of scenes, equipment, actors, and shooting crew, the cost of using resources such as scenery, costumes and professional digital cameras, and also fixed cost incurred during a movie shooting.

Constraint (3.2) guarantees that each scene has a starting point of 0, and at most, one scene can be shot as the starting scene.

Constraint (3.3) guarantees that each scene has an endpoint T , and at most, one scene can be shot as the final scene.

Constraint set (3.4) ensures that only one scene can be shot after each scene.

Constraint set (3.5) ensures that only one scene can be shot before each scene.

Constraint set (3.6) guarantees that if scene j is the next scene of scene i , then scene i cannot be the next scene of scene j and vice versa.

Constraint set (3.7) ensures that if scene j is the next scene of scene i , then starting time of scene j should be greater than or equal to the completion time of scene i .

Constraint set (3.8) guarantees the precedence relation between scene i and scene j . If scene i is the predecessor of scene j , starting time of scene j should be greater than or equal to the completion time of scene i .

Constraint set (3.9) ensures that if scene i has a time window, starting time of scene i should be greater than or equal to the earliest starting time of scene i .

Constraint set (3.10) ensures that if scene i has a time window, the completion time of scene i should be less than or equal to the latest completion time of scene i .

Constraint set (3.11) ensures that if actor k has an unavailable time period and scene i is shot before the ending of unavailable time of actor k , the completion time of scene i should be less than or equal to the beginning of the unavailable time of actor k .

Constraint set (3.12) ensures that if actor k has an unavailable time period and scene i is shot after the ending of unavailable time of actor k , starting time of scene i should be greater than or equal to the ending of unavailable time of actor k .

Constraint set (3.13) guarantees that if scene i requires resource h , latest completion time of the scenes that require resource h , should be greater than or equal to the completion time of scene i .

Constraint set (3.14) guarantees that if scene i requires resource h , starting time of the scenes that require resource h , should be less than or equal to the starting time of scene i .

Constraint sets (3.15) and (3.16) ensure that, if all scenes in which actor k plays are shot after the ending of unavailable time or before the beginning of unavailable time of actor k , there will be no payment reduction for actor k . If, at least one of the scenes in which actor k plays, is shot either after the ending of unavailable time or before the beginning of unavailable time of actor k , there will be a payment reduction for actor k .

Constraint set (3.17) gives the starting time of scene i in which actor k plays.

Constraint set (3.18) gives the completion time of the last scene in which actor k plays.

Constraint set (3.19) gives the starting time of the first scene in which actor k plays.

Constraint set (3.20) gives the first scene's starting time according to all scenes' starting times.

Constraint set (3.21) gives the last scene's completion time according to all scenes' completion times.

Constraint (3.22) gives the duration of the movie's shooting from the starting time of the first scene to the completion time of the last scene.

Constraint set (3.23) guarantees that the completion time of all scenes cannot exceed the due date of the movie's shooting.

Constraint sets (3.24) – (3.28) are nonnegativity restrictions.

Constraint sets (3.29) – (3.31) are binary restrictions.

CHAPTER 4

PSO ALGORITHM

The MILP model is an exact solution method to solve the MSSP as addressed in this study. However, as discussed in Section 5.2 in the following chapter, solving this combinatorial problem requires long computational times as the problem size increases. Therefore, to overcome this challenge, a population-based meta-heuristic that uses a PSO has been developed to obtain the optimal or near-optimal schedule with the same objective of minimizing the total cost of a movie shooting. The reason why the PSO algorithm is selected is that it has a simple structure and fewer parameters to adjust than the other meta-heuristic algorithms have. Moreover, it is easier to implement and also computationally efficient in typical combinatorial optimization problems.

This chapter has been organized as follows:

- In Section 4.1, a brief information on the PSO algorithm is provided.
- In Section 4.2, variants of the PSO algorithm in the literature are introduced.
- In Section 4.3, the proposed PSO algorithm is explained in detail.

4.1 Brief Information about the PSO Algorithm

PSO algorithm is first introduced by Kennedy and Eberhart (1995) with the inspiration of the social behavior of birds flocking and fish schooling. PSO is one of the latest population-based optimization methods and has been applied to a wide range of combinatorial problems, such as scheduling, and supply chain network design, to operate in the continuous space.

PSO algorithm starts by initializing a population called a “swarm” and searches for the optimal solution by updating individuals of the whole swarm. Each solution is

called a “particle” in a PSO algorithm. These particles move around the search space as the algorithm runs for a specified number of iterations. Therefore, increasing the number of particles allows you to explore new areas to find diverse solutions. Each particle has a position at the beginning and flies around the search space with a velocity as the algorithm goes to the posterior iterations. The velocity and position of each particle are updated according to the particle’s own experience and the other particles’ experience in the swarm iteratively. Then, particles are evaluated by the fitness function value at each iteration. At each iteration, the particles focus on the search in a local region by exploiting their own experience. The core operations are revising the velocity and position for each particle with Formulas (4.1) and (4.2), respectively. Formula (4.1) is utilized to calculate a particle’s new velocity from the particle’s previous velocity and the distances from its current position to its local best and the global best positions. Formula (4.2) is used to calculate a particle’s new position by using the new velocity value and the position at iteration $r - 1$.

$$V_i^r = V_i^{r-1} + c_1 rand_1 (P_i^{r-1} - X_i^{r-1}) + c_2 rand_2 (PG^{r-1} - X_i^{r-1}) \quad (4.1)$$

$$X_i^r = V_i^r + X_i^{r-1} \quad (4.2)$$

$P(i)^{r-1}$ represents the local (personal) best of the i^{th} particle encountered at iteration $r - 1$ while PG^{r-1} denotes the global best among all particles of the swarm achieved until iteration r . c_1 and c_2 are positive acceleration constants that imply learning factors. The cognitive learning effect is expressed with c_1 while social learning is described with c_2 . $rand_1$ and $rand_2$ are random numbers selected from the interval $[0, 1]$.

Shi and Eberhart (1998) improve the efficiency of the algorithm by introducing a linearly varying inertia weight w over the generations. The velocity updating is reformed, as shown in Formula (4.3).

$$V_i^r = w^r V_i^{r-1} + c_1 rand_1 (P_i^{r-1} - X_i^{r-1}) + c_2 rand_2 (PG^{r-1} - X_i^{r-1}) \quad (4.3)$$

where w^r is given by:

$$w^r = w_{\max} - \frac{w_{\max} - w_{\min}}{itermax} iter \quad (4.4)$$

w^r is a positive inertia weight used to control the effect of the previous velocities over the current velocity. This parameter can be introduced as a constant or time-decreasing manner with Formula (4.4) where $w_{\max}$ and $w_{\min}$ are generally chosen as 0.8 and 0.4, respectively, and $iter$ is the current iteration number, and $itermax$ is a predefined maximum number of iterations.

Besides the original PSO algorithm, Kennedy and Eberhart (1997) also develop a discrete version of the PSO algorithm to solve discrete optimization problems. There are two essential differences regarding the characteristics of the original (continuous) PSO and the discrete version of it. Firstly, the particle is described with binary variables. Secondly, the velocity is converted to the change of probability, which denotes the chance of the binary variable getting one or zero value. In a discrete PSO, position vectors and velocity vectors of the particles are represented in an n -dimension space as:

$$\begin{aligned} X(i)^r &= [x(i, 1)^r, x(i, 2)^r, \dots, x(i, n)^r], & x(i, n)^r &\in \{0, 1\} \\ V(i)^r &= [v(i, 1)^r, v(i, 2)^r, \dots, v(i, n)^r], & v(i, n)^r &\in R, \text{ respectively.} \end{aligned}$$

As in the original (continuous) PSO, velocity is updated with Formula 4.3. The upper bound ($v_{\max} = +4.0$) and lower bound ($v_{\min} = -4.0$) for the velocities are specified to prevent the corresponding probability values from approaching 1 or 0.

Then, by the sigmoid function introduced in Formula 4.5, the updated velocity values are converted from real numbers to the changes of probability values between [0,1]:

$$s[v(i, j)^r] = \frac{1}{1 + e^{-[v(i, j)^r]}} \quad (4.5)$$

where $v(i, j)^r$ is the velocity of element j in particle i at iteration r . The higher probability of the element in a particle is more likely to be 1 and vice versa. For instance, $s[v(i, 2)^r] = 0.2$, there is a chance that element 2 of particle i will be 1.

To sum up, considering the information mentioned above, the general process of the PSO algorithm is given below:

Initialize random velocities and positions of the particles of a swarm.

While the stopping criterion is not met, do:

For each particle i do

Evaluate the fitness of each particle.

Update the personal best of each particle:

Update global best among the particles of the swarm.

Update the velocity and position of each particle.

End for

End while

4.2 Related Studies on the PSO Algorithm

MSSP under study has similar characteristics with some problems as: single machine scheduling and project scheduling, sequential ordering, traveling salesman, assembly line balancing, and task assignment problems. Thus, these related problems and their PSO heuristic-based solution methodologies are reviewed in detail for both continuous and discrete versions of the PSO algorithm in this section.

4.2.1 Studies Using Continuous Version of the PSO Algorithm

Tasgetiren et al. (2004) apply the PSO algorithm for a single machine scheduling with total weighted tardiness problem where each particle in the swarm denotes the permutation of jobs to be scheduled.

As a result, particle i at iteration r is expressed as a point with dimension n for n many jobs with a position vector:

$$X(i)^r = \{x(i, 1)^r, x(i, 2)^r, \dots, x(i, n)^r\}.$$

The velocity of particle i is represented by a vector:

$$V(i)^r = \{v(i, 1)^r, v(i, 2)^r, \dots, v(i, n)^r\}.$$

Since each particle has a continuous position value, the smallest position value (SPV) rule is utilized to find a sequence of jobs in a particle. Tasgetiren et al. (2007) introduce the same heuristic rule for makespan and total flowtime minimization in the permutation flowshop sequencing problem.

Similarly, Jayanthi and Anusuya (2017) deal with a single machine total weighted earliness and tardiness minimization by employing the PSO algorithm. The particles are represented as a permutation of the jobs, and the job sequence is organized according to the smallest position value (SPV) rule, as well.

Generally, the penalty function method has been utilized during the past decade for constrained optimization problems. However, due to its drawback regarding the decision on appropriate penalty factors, He and Wang (2007) develop a hybrid PSO algorithm with a feasibility-based rule to handle constraints regardless of the characteristics of the problem addressed. The employed feasibility rule is stated below:

- Choosing any feasible solution instead of any infeasible solution.
- Choosing the solution which gives a better objection function value among two feasible solutions.
- Choosing the solution which gives the smaller constraint violations among two infeasible solutions.

Tchomte and Gourgand (2009) propose an extension of the PSO algorithm by introducing a new displacement rule for particles to balance both intensification and diversification processes. Owing to this new rule, each particle moves from the

current position, $X_{i,t}$, to the next position, $X_{i,t+1}$ by visiting intermediate positions $S_{i,t}$ and $T_{i,t}$ so that intensification is satisfied more than the original PSO algorithm.

In the sequential ordering problem, the PSO algorithm is utilized with a local search procedure by Anghinolfi et al. (2011). Precedence relations are fulfilled while minimizing the total cost. The devised PSO algorithm may distort the precedence relationships when velocities and positions are updated. As a result, a fixing procedure is developed and applied to satisfy the precedence relationships between scenes.

Naderi et al. (2014) propose a PSO algorithm using an acceptance criterion (probability of $e^{-\Delta X/T}$) to increase diversification capability. They define P_1^{best} and P_2^{best} rather than considering only P^{best} to avoid stagnation in local optima. In addition, a local search procedure is applied, relying on a tabu search algorithm.

4.2.2 Studies Using Discrete Version of the PSO Algorithm

Salman et al. (2002) apply a different version of the discrete PSO. A task assignment problem is solved in which M tasks are assigned to N processors. Each particle is represented as a vector of M tasks, and each task in a particle is indicated as an integer value corresponding to the number of its processor. Since real values can be obtained after updating the velocity and positions of the particles, the signs ($\pm$) and the fractional parts are removed to get integer values indicating the numbers of processors. Therefore, each particle flies in an M -dimensional search space with a discrete set of possible values limited by the number of processors, N .

Tasgetiren et al. (2007) study the traveling salesman problem using the discrete PSO. Particles are indicated with both permutations of clusters n_j , node π_j , and distance $d_{\pi_j, \pi_{j+1}}$ from node π_j to π_{j+1} .

Zhang et al.(2006) solve a multi-mode project scheduling to minimize the project duration by using the PSO method to minimize total project duration. In a particle, each dimension or element corresponds to the priority of that activity.

Liao et al. (2007) propose a different form of a discrete PSO for a flowshop scheduling problem. Job-to-position representation is used, and particles are described with a position vector where $x_{ijk} = 1$, if job j is placed at position k of the sequence and 0 otherwise. The Largest Position Value (LPV) rule is utilized for the changes of probabilities to obtain a job sequence. Low et al. (2010) use job-to-position depiction for the particles and employ the same rule in their modified PSO algorithm to solve the single machine scheduling problem with periodic maintenance.

Petropoulos and Nearchou (2011) solve an assembly line balancing problem with a PSO method. Real-valued updated positions of the particles are converted to the sequence of jobs according to their priorities and predecessor relations.

Jarboui et al. (2008) develop an extension of a discrete PSO to solve the multi-mode resource-constrained project scheduling problem. Each particle is represented with mode numbers. For instance, $x_{i4}^t = 2$ means task 4 is assigned to mode 2 in particle i . An intermediate dummy variable (Y_i^t) is used to convert continuous velocity and position values to integer values that correspond to the mode number and vice versa. Khalilzadeh et al. (2012) cope with minimizing the total costs of resources in multi-mode resource-constrained project scheduling problem and apply the extended PSO devised by Tchomte and Gourgand (2009) with some additional improvement heuristics to prioritize activities and improve the solution quality.

Moreover, Koulinas et al. (2014) and Joy et al. (2016) deal with the resource-constrained project scheduling problem, and represent the particles by the priority values of the activities. A serial scheduling generation scheme is used to build a feasible schedule fulfilling the precedence relations. Then, solutions are evaluated according to the makespan values of the solutions.

4.3 The Proposed Discrete PSO Algorithm

As a novel population-based algorithm, PSO has been used for a wide variety of applications, especially for unconstrained optimization problems. Since our problem is a scheduling problem under some constraints about the actors and scenes, an extended version of the discrete PSO algorithm is developed in this section.

The goal of such a problem is finding the optimum schedule of both scenes and actors to minimize the total cost of movie shooting. Therefore, it is crucial to have a solution representation related to the problem domain. An extended version of the discrete PSO algorithm is established to find optimal/near-optimal solutions under some constraints, such as precedence relations among scenes, blocking time periods of actors, and time windows of scenes.

For constrained optimization problems, the penalty function method has been the most popular technique to handle constraints so far. However, this method has some drawbacks in choosing suitable penalty parameters not to weaken the efficiency of the technique. Therefore, for the MSSP, the feasibility-based rule is applied in the developed discrete PSO algorithm to be independent of choosing penalty factors, as in Sun et al. (2009) and He and Wang (2007).

In our MSSP, scenes-to-position representation for the particles is proposed for effective problem mapping. In other words, each particle corresponds to a permutation of scenes which gives a solution to the problem. n many scenes are positioned in a set of m particles, as shown below using an example:

$$X(i)^r = [x(i, 1, 1)^r, x(i, 2, 2)^r, \dots, x(i, n, n)^r], \quad x(i, j, k)^r \in \{0, 1\},$$

where $x(i, j, k)^r = 1$, if scene j is placed in k th position of the sequence i at iteration r , and 0 otherwise.

The corresponding velocity vector of each particle is defined as:

$V(i)^r = [v(i, 1, 1)^r, v(i, 2, 2)^r, \dots, v(i, n, n)^r]$, $v(i, j, k)^r \in R$, in the range of $[-V_{min}, +V_{max}]$.

If $v(i, j, k)^r$ is smaller than $-V_{min}$, then the velocity is set to $-V_{min}$.

If $v(i, j, k)^r$ is larger than $+V_{max}$, then the velocity is set to $+V_{max}$.

The general framework of the proposed PSO algorithm is given below:

- Phase 1: Initialization
- Phase 2: PSO
 - Phase 2.1: Repairing
 - Phase 2.2: Improvement

The PSO algorithm is started by generating particles in terms of both position and velocity vectors. In general, initial particles' positions and velocities are generated randomly. However, randomly generated solutions may cause inefficiency in the algorithm. In addition, these randomly generated particles may fail to cover the search space completely. As a result, particles may stay away from the global optimum and be trapped in local optima. Therefore, in the initialization phase, feasible particles that satisfy the constraints regarding precedence relations among scenes, unavailability time periods of actors and time windows of scenes are only generated instead of random generation. In the PSO phase, the velocity and position values of the particles are updated. In addition, personal (local) and the global best values are kept for the posterior iterations. In the repairing phase, scene sequences are repaired regarding the precedence relations. In the improvement phase, schedules are improved to get a better objective function value. In Sections 4.3.1 and 4.3.2, all phases of the algorithm are described in detail.

4.3.1 Phase 1: Initialization

Before applying the proposed PSO algorithm, initial feasible particles are generated, as indicated in the following steps.

Step I.1: The number of feasible particles to be used in the PSO algorithm is taken as input from the user.

Step I.2: A prespecified number of particles (i.e., sequence of scenes) are generated according to the precedence relations among scenes only.

Step I.3: The starting and completion times of each scene in the sequence are calculated to get a schedule. While scheduling the scenes, the unavailable time periods of actors, earliest starting and latest completion times of scenes (i.e., time windows) are taken into consideration. Thus, the generated sequences (particles) are all feasible.

4.3.2 Phase 2: PSO

After obtaining feasible particles in the initialization step, the particle swarm optimization algorithm is started and continues for the predefined number of iterations. A flowchart of the PSO can be seen in Appendices A-C. Furthermore, steps in the algorithm are explained in detail as follows.

Step P.1: Initial position vectors are created for each feasible particle i . Position vectors, $X(i)^r$, contain $x(i, j, k)^r$ elements indicating whether scene j is in k^{th} position in particle i at r^{th} iteration or not. $x(i, j, k)^r$ elements take an integer value of 1 if scene j is in k^{th} position in particle i or take an integer value of 0 if scene j is not in k^{th} position in particle i at iteration r . For instance, if scene 3 is in the 2^{nd} position in particle 4 obtained in Step P.1, $x(4, 3, 2)^r = 1$ and other $x(4, 3, k)^r$ values for $k \neq 2$ take a value of 0 at iteration r .

Step P.2: Initial velocity vectors $V(i)^r$ are created for each feasible particle i by using Formula 4.6 given below. Velocity vectors, $V(i)^r$ contain $v(i, j, k)^r$ elements taking real values in the range of $[-V_{min}, +V_{max}]$.

$$v(i, j, k)^r = V_{min} + rand() (V_{max} - V_{min}) \quad (4.6)$$

where V_{min} and V_{max} values are -4, and +4, respectively. $rand(.)$ is a random number selected from the interval $[0, 1]$.

Step P.3: Fitness value corresponding to the objective function value is calculated for each feasible particle based on Formulas (4.7-4.10). Objective function value includes the wage of actors, transfer cost of scenes, actors and shooting crew, cost of using resources, and fixed cost incurred during movie shooting.

$$\text{Wage of actors: } \sum_{k \in A} W_k \cdot (cls_k - minstk_k) - \sum_{k \in AU} W_k \cdot pr_k \cdot (EU_k - BU_k) \quad (4.7)$$

$$\text{Transfer cost: } \sum_{i \in S} \sum_{j \in S} CM_{ij} \cdot x_{ij} \quad (4.8)$$

$$\text{Resource usage cost: } \sum_{h \in R} RC_h \cdot (lct_h - est_h) \quad (4.9)$$

$$\text{Fixed cost: } FC \times duration \quad (4.10)$$

Step P.4: Personal (local) best position vector $P(i)^r$ is created at the beginning of iterations ($r = 0$). At iteration 0, the personal best position vector is equal to the position vector itself for each particle i .

Step P.5: The global best position vector $PG()^r$ is created among all particles with the minimum cost.

Step P.6: Iteration counter is incremented.

Step P.7: The inertia weight is updated according to Formula 4.11, as shown below:

$$w^r = w_{max} - \frac{w_{max} - w_{min}}{itermax} iter \quad (4.11)$$

w^r is a positive inertia weight used to control the effect of the previous velocities on the current velocity. This parameter can be introduced as a constant or in a time-decreasing manner with Formula (4.11). w_{max} and w_{min} are generally chosen as 0.8 and 0.4, respectively. $iter$ is the current iteration counter, and $itermax$ is the predefined maximum number of iterations.

Step P.8: The velocity vectors are updated based on Formula 4.12, as shown below:

$$\begin{aligned} v(i, j, k)^r = & w^r v(i, j, k)^{r-1} + c_1 rand_1(p(i, j, k)^{r-1} - x(i, j, k)^{r-1}) \\ & + c_2 rand_2((pg)^{r-1} - x(i, j, k)^{r-1}) \end{aligned} \quad (4.12)$$

Step P.9: Velocity values are converted to the change of probability values that indicate the chance that scene j will be positioned in position k . Probability values lying between $[0, 1]$ are obtained with the sigmoid function in Formula 4.13:

$$s[v(i, j, k)^r] = \frac{1}{1 + e^{-[v(i, j, k)^r]}} \quad (4.13)$$

Step P.10: To avoid being trapped in local optima, a random number between $[0, 1]$ is used to decide on placing scene j to k^{th} position in particle i at iteration r .

If $rand() < s[v(i, j, k)^r]$:

$$x(i, j, k)^r = 1$$

Else:

$$x(i, j, k)^r = 0$$

In this way, new positions are obtained for all scenes in particle i . As a result, new child particles are created from each parent particle i .

Step P.11: Feasibility is checked regarding precedence relations for each new child particle. If a new child particle is not feasible, a repairing procedure is applied to get a new sequence of scenes regarding precedence relations. If a new particle is already feasible, the sequence of scenes is not changed.

Step P.12: Each new child particle is scheduled to find starting and completion times of all scenes in the sequence. The earliest starting time of scenes, and the beginning and ending of unavailability times of actors are considered while scheduling scenes in each new child particle.

Step P.13: Feasibility is checked regarding the latest completion time of scenes in each new child particle. If the completion time of a scene is greater than the latest completion time of the corresponding scene, this new child particle is considered an infeasible one. As a result, the child particle is ignored, and the corresponding parent particle is preserved. If the completion time of a scene is less than or equal to the latest completion time for the corresponding scene, the new child particle is considered a feasible one, and the new child particle replaces the parent particle.

Step P.14: Improvement procedure is applied to feasible particles (either new feasible child particles or feasible parent particles) according to the rule in the previous step.

Step P.15: Fitness values are calculated for both improved new feasible child particles and improved feasible parent particles using the objective function as stated in Formulas 4.7–4.10 in Step P.3.

Step P.16: The fitness function value of the improved new feasible child particle and each corresponding parent particle are compared against each other. If the parent particle gives the minimum cost compared to the improved new feasible child particle, the parent particle is preserved, and its personal best position vector is not changed. If the improved new feasible child particle gives a better cost value than the parent particle, the child particle replaces the parent particle, and position and velocity vectors are created.

Step P.17: For the improved new feasible child particles replacing the parent particle, position and velocity vectors are created; personal best vectors are updated.

Step P.18: The global best vector at iteration r , $PG()^r$, is created among the particles at hand.

Step P.19: The global best vector found at iteration r , $PG()^r$, is compared to the one, $PG()^{r-1}$ found at iteration $r - 1$. If the global best vector found at iteration r gives a better fitness function value than the one found at iteration $r - 1$, the global best vector is updated with the one found at iteration r . Otherwise, the global best vector is not changed for the next step.

Step P.20: The stopping criterion is checked to determine whether it is satisfied or not. If it is satisfied, the scene sequence is obtained from the current global best vector. Otherwise, the next iteration is followed.

In the proposed PSO algorithm, a prespecified number of iterations are used as the stopping criterion. The algorithm is run until the prespecified number of iterations are completed.

4.3.2.1 Phase 2.1: Repairing

If the generated particles do not satisfy the precedence relations at Step P.11, the repairing procedure is applied to get a feasible sequence of scenes. Steps for the repairing procedure are given as follows:

Step R.1: The repairing procedure is started with the scene in position $k = n$ (last in sequence).

Step R.2: If the scene in position k has a successor, the current position of the successor scene in the sequence is found and this value is set to a variable called “helpposition”. If the scene in position k does not have a successor, the repairing

procedure is started for the previous scene that is in position $k = k - 1$ with Step R.2.

Step R.3: If “helpposition” is smaller than position k , the successor scene is removed from the sequence and the repairing procedure continues with Step R.4. If the helpposition is greater than the position of the scene in position k , the repairing procedure is started for the previous scene that is in position $k = k - 1$ with Step R.2.

Step R.4: The scenes in positions in the range of $[\text{helpposition}+1, n]$ are shifted to the left. In other words, scenes that are positioned after the successor scene are shifted to the left.

Step R.5: The removed successor scene is inserted into position $k = n$. (the last position of the sequence), and the repairing procedure starts with Step R.1.

Step R.6: The repairing procedure ends after checking the feasibility of the precedence relations among scenes.

The flowchart of the repairing procedure is given in Appendix D.

4.3.2.2 Phase 2.2: Improvement

At the end of Step P.13, an improvement procedure is applied to get a better objective function value. The improvement process has two parts: Control 1 and Control 2. If Control 1 cannot be used, Control 2 is followed.

Control 1:

With the help of Control 1, the starting time of the first scene in the sequence is postponed to the latest ending of the unavailable time of all actors if there is no violation of the latest completion time for all scenes. Related steps of Control 1 are explained below:

Step C1.1: The latest ending of the unavailable time of actors is found.

Step C1.2: Starting time of the first scene is set to the latest ending of the unavailable times of actors.

Step C1.3: The following scenes in the sequence are rescheduled to obtain the starting and completion times.

Step C1.4: The latest completion times of scenes and their new completion times are compared against each other for all scenes in the sequence. If the new completion times are less than or equal to the latest completion times of all corresponding scenes, new starting and completion times of scenes are updated as calculated in Step C1.3. If the completion time of at least one scene is greater than the latest completion time of the corresponding scene, feasibility is violated, so old starting and completion times of all scenes are preserved. As a result, Control 1 is ended, and Control 2 is followed.

Control 2:

Control 2 is applied to get a sequence of scenes with less idle time between them. For a particle with n scenes, Control 2 is started from the scene in position $n - 1$.

Step C2.1: The completion time of the scene in position $n - 1$ is compared with the starting time of the scene in position n .

Step C2.2: If the completion time of the scene in position $n - 1$ is equal to the starting time of the scene in position n , the completion time of the scene in position $n - 2$ is compared with the starting time of the scene in position $n - 1$. Step C2.2 is repeated until completing the control of scene in position 1 in the sequence.

Step C2.3: If the completion time of the scene in position $n - 1$ is not equal to the starting time of the scene in position n , after checking the latest completion time and unavailability time of actors, the completion time of the scene in position $n - 1$ is tried to be equalized to the starting time of the scene in position n .

Step C2.4: If the completion time of the scene in position $n - 1$ is changed in Step C2.3, the new starting time of that scene is calculated. In addition, the completion

and starting times of previous scenes are calculated again. Then, Steps C2.2 and C2.3 are repeated for the previous scenes to compare their completion times with the starting times of their following scenes.

In addition, the flowchart of the improvement procedure is given in Appendices E and F.

CHAPTER 5

COMPUTATIONAL EXPERIMENTS

The performance of the proposed solution methods, the MILP model and the PSO algorithm is evaluated and reported in this chapter based on the computational experiments.

In Section 5.1, the graph structures of precedence relationships that are suitable for MSSP are introduced. In Section 5.2, by using the generated problem instances and a real film script instance, the performance of the PSO algorithm is compared with the MILP model that is solved by the CPLEX solver. In Section 5.3, the effects of the number of particles and the number of iterations on the PSO performance are examined. In Sections 5.4 and 5.5, the overall performances of the MILP model and PSO algorithm are evaluated regarding all runs for the problem instances, respectively.

5.1 Graph Types of Precedence Relationships

While developing an optimization algorithm, it is vital testing the algorithm with a deep variety of test problems to examine its strengths and weaknesses. Prot and Bellenguez-Morineau (2018) emphasize that the graph structure of a precedence relationship network of jobs has an essential effect on the complexity of scheduling problems. In addition, they introduce precedence constraints that have already been studied and modeled by two types of directed acyclic graphs in scheduling theory: series-parallel (SP) and opposing forest (OF). Due to the nature of precedence relations in MSSPs, these two directed acyclic graphs seem to be appropriate and hence are constructed for the precedence relationships among the scenes as follows:

- Series parallel (SP) graph: This type of graph consists of two vertices joined by two parallel edges starting from one node.
- Opposing forest (OF) graph: This type of graph is a collection of intrees and outtrees.

As a result, these two types of precedence relation graphs are constructed so that the performance of the proposed PSO algorithm is tested by a broad range of test problems with different complexity levels. Furthermore, it can be concluded from Appendices G, H, and I that the precedence relations among scenes in the real film script instance used in evaluating the performance of the proposed PSO algorithm are of the OF-graph type.

5.2 Performance Comparison of the MILP Model and PSO Algorithm

The solutions obtained by the PSO algorithm are analyzed regarding objective function values Z_{PSO} , computational times, and percent deviation from the MILP model solutions, Z_{MILP} . The related percentage deviation PD is expressed with Formula 5.1.

$$PD = \frac{Z_{PSO} - Z_{MILP}}{Z_{MILP}} \times 100 \quad (5.1)$$

The precedence relations among the scenes are represented in two different graph types, SP and OF, so that the effects of graph type, if any, can be observed on the solution quality and computational time. The problem instances are generated based on the time window and blocking time constraints as follows:

- (i) problem instances without time window and blocking time constraints,
- (ii) problem instances with blocking time constraint only,
- (iii) problem instances with time window constraint only,
- (iv) problem instances with blocking time and time window constraints.

The problem instances that include blocking time and/or time window constraints are coded by B and/or T, respectively.

To begin with, a problem instance is generated, ignoring the two constraints regarding blocking time and time window. Then, five problem instances are defined with blocking time constraints, and five problem instances are defined with time window constraints. Finally, the problem instances, including both blocking time and time window constraints, are generated. Time window and blocking time constraints are defined for 20% of the scenes and 20% of the actors in the problem instances. For instance, two scenes have time window constraints and two actors have blocking time constraints in the problem instance with 10 scenes and 10 actors. Therefore, 16 problem instances are generated in Section 5.2.1. Moreover, a real film script instance is introduced in detail in Section 5.2.2.

5.2.1 Generated Problem Instances

Although there are several parameters related to a film and its scenes, the numbers of scenes and actors are the major ones that determine the problem size. As a result, in this section, five different problem sizes are considered in problem generation to evaluate the PSO performance against the solutions obtained by the CPLEX solver for the MILP model. The different-sized problems are as follows:

- 10 scenes/10 actors (10S/10A),
- 15 scenes/10 actors (15S/10A),
- 20 scenes/10 actors (20S/10A),
- 25 scenes/10 actors (25S/10A),
- 25 scenes/15 actors (25S/15A).

The precedence relationships of these problems are generated based on both SP-graph and OF-graph types.

The problem instances are solved by the CPLEX solver for the optimum solution and also by the PSO algorithm with 30 particles for 20 iterations. The PSO

algorithm is implemented in Python programming language, and the problems are solved on a PC with 8 GB RAM, 11th Gen Intel® Core™ i5-1135G7 @ 2.40 GHz, Windows 11 version.

5.2.1.1 Problem Instances with 10 Scenes and 10 Actors

In this section, the test problems involve 10 scenes and 10 actors. Firstly the SP-graph and secondly the OF-graph are structured for the 10 scenes' precedence relationships as shown in Figures 5.1 and 5.2, and the corresponding solutions are provided in Tables 5.1. and 5.2.

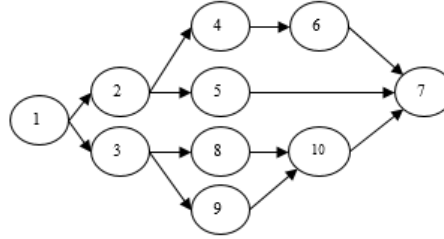


Figure 5.1. SP-Graph for 10S/10A

When we analyze Table 5.1 and Figure 5.1, we can make the following observations for the problem instances with 10 scenes and 10 actors with precedence relationships of SP-graph type:

- The problem instance without constraints for blocking times of actors and time windows of scenes can be solved in 0.14 seconds by the CPLEX solver and 11.65 seconds by the PSO algorithm.
- For the CPLEX solver, by including only blocking time, the average computational time increases, whereas by including only time window constraint, the average computational time decreases. This shows that imposing only blocking time constraints in the problem instances increases the complexity of the problems. In contrast, only time window constraints shrink the solution space and enable the problems to be solved quickly.

Table 5.1 MILP and PSO Solutions for 10S/10A with SP-Graph

<i>Graph Type</i>	<i>Problem Instance</i>	<i>Z_{MILP}</i>	<i>Optimality Gap</i>	<i>CPU (sec)</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>
SP	10S/10A	48985	0.00%	0.14	48985	11.65	0
	10S/10A/B-1	48985	0.00%	0.17	48985	11.2	0
	10S/10A/B-2	48985	0.00%	0.19	48985	11.68	0
	10S/10A/B-3	48560	0.00%	0.27	48985	11.64	0.88
	10S/10A/B-4	48670	0.00%	0.24	48785	14.04	0.24
	10S/10A/B-5	48690	0.00%	0.31	48985	10.76	0.61
	Average of 10S/10A/B		0.00%	0.24		11.86	0.34
	10S/10A/T-1	48985	0.00%	0.05	48985	24.74	0
	10S/10A/T-2	49650	0.00%	0.17	49660	22.01	0.02
	10S/10A/T-3	51660	0.00%	0.03	51660	16.62	0
	10S/10A/T-4	54485	0.00%	0.01	54485	49.06	0
	10S/10A/T-5	52630	0.00%	0.02	52630	19.41	0
	Average of 10S/10A/T		0.00%	0.06		26.37	0
	10S/10A/B/T-1	48985	0.00%	0.03	48985	33.21	0
	10S/10A/B/T-2	51250	0.00%	0.03	51250	21.74	0
	10S/10A/B/T-3	51235	0.00%	0.03	51235	17.32	0
	10S/10A/B/T-4	67545	0.00%	0.01	67545	61.3	0
	10S/10A/B/T-5	51660	0.00%	0.02	51660	19.37	0
	Average of 10S/10A/B/T		0.00%	0.02		30.59	0

- The lowest average computational time by the CPLEX solver is observed by imposing both blocking time and time window constraints.
- By the PSO algorithm, as the constraints are imposed in the problem instances, the average solution times increase, and the highest computational time is observed for the problem instances that have both blocking time and time window constraints.
- 13 out of 16 problem instances are solved to optimality; and only 3 instances with blocking time are solved with an average deviation of 0.34 %.

When we analyze Table 5.2 and Figure 5.2, we can make the following observations for the problem instances with 10 scenes and 10 actors with precedence relationships of the OF-graph type:

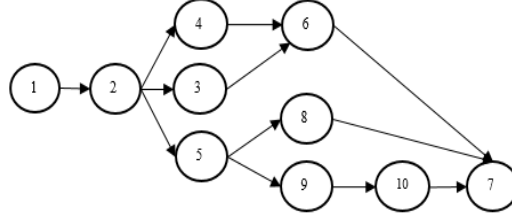


Figure 5.2. OF-Graph for 10S/10A

- The problem instance without constraints for blocking times of actors and time windows of scenes can be solved in only 0.17 and 13.66 seconds by the CPLEX solver and the PSO algorithm, respectively.
- By both the CPLEX solver and PSO algorithm, the average computational time decreases when including only blocking time constraints.
- By the CPLEX solver, the average computational time decreases when including only blocking time constraints and also only time window constraints. In addition, the lowest average computational time is observed when including both blocking time and time window constraints simultaneously. This means that as the problem instances get more constrained, the solution space is reduced, and as a result, the average computational time decreases by the CPLEX solver.
- By the PSO algorithm, when only blocking time constraints are imposed in the problem instances, the average computational time decreases. However, the average computational time increases when only time window constraints are imposed in the problem instances. This shows that time window constraints are more challenging for the PSO algorithm.

- The highest average computational time of 21.51 seconds is observed in the problem instances that have both blocking time and time window constraints by the PSO algorithm.

Table 5.2 MILP and PSO Solutions for 10S/10A with OF-Graph

Graph Type	Problem Instance	Z_{MILP}	Optimality Gap	CPU (sec)	Z_{PSO}	CPU (sec)	PD
OF	10S/10A	52100	0.00%	0.17	52100	13.66	0
	10S/10A/B-1	52055	0.00%	0.09	52100	11.65	0.09
	10S/10A/B-2	52100	0.00%	0.11	52100	11.04	0
	10S/10A/B-3	51795	0.00%	0.16	52120	10.27	0.63
	10S/10A/B-4	52055	0.00%	0.14	52100	10.18	0.09
	10S/10A/B-5	51130	0.00%	0.17	52120	10.36	1.94
	Average of 10S/10A/B		0.00%	0.13		10.7	0.41
	10S/10A/T-1	52100	0.00%	0.03	52100	21.44	0
	10S/10A/T-2	52255	0.00%	0.03	52255	12.43	0
	10S/10A/T-3	53165	0.00%	0.02	53165	17.52	0
	10S/10A/T-4	54185	0.00%	0.02	54185	24.9	0
	10S/10A/T-5	52660	0.00%	0.03	52780	24.78	0.23
	Average of 10S/10A/T		0.00%	0.03		20.21	0.05
	10S/10A/B/T-1	53085	0.00%	0.03	53235	20.48	0.28
	10S/10A/B/T-2	59770	0.00%	0.02	60045	16.12	0.46
	10S/10A/B/T-3	52990	0.00%	0.02	52990	17.52	0
	10S/10A/B/T-4	60690	0.00%	0.03	60690	27.94	0
	10S/10A/B/T-5	51690	0.00%	0.02	51810	25.47	0.23
	Average of 10S/10A/B/T		0.00%	0.02		21.51	0.19

- The highest computational time of 27.94 seconds is observed for problem instance 10S/10A/B/T-4 that has both blocking time and time window constraints by the PSO algorithm.
- 8 out of 16 problem instances are solved to optimality by the PSO algorithm.
- The highest average deviation of 0.41 % is obtained when only blocking times for actors are imposed in the problem instances.

To sum up, for the problem instances with 10 scenes/10 actors, it can be inferred that the highest average computational time is observed as both blocking time and time window constraints are imposed in the problem instances, regardless of the precedence relations graph types, by the PSO algorithm.

The lowest average computational time is observed as both blocking time and time window constraints are involved in the problem instances, regardless of the precedence relations graph types, by the CPLEX solver. This indicates that as constraints are included in the problem instances, the solution space is reduced, and as a result, the average computational time decreases by the CPLEX solver.

For the problem instances in the SP-graph type, as including only blocking time constraints, the average computational time increases, whereas it decreases in the problem instances in the OF-graph type by the CPLEX solver. Therefore, it can be inferred that problem instances with only blocking time constraints in the OF-graph type are solved more easily than ones with only blocking time constraints in the SP-graph type by the CPLEX solver.

5.2.1.2 Problem Instances with 15 Scenes and 10 Actors

In this section, 15 scenes and 10 actors are involved in the problems. SP-graph and OF-graph types are constructed to describe the precedence relations among scenes. Figures 5.3 and 5.4 show the SP-graph and OF-Graph types, respectively. The solutions are provided in Tables 5.3 and 5.4 for these graph types.

When we analyze Table 5.3 and Figure 5.3, we can make the following observations for the problem instances with 15 scenes and 10 actors with precedence relationships of the SP-graph type:

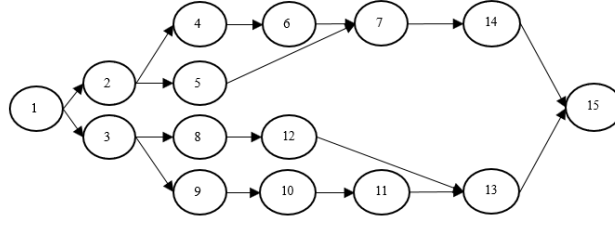


Figure 5.3. SP-Graph for 15S/10A

- It takes 13.88 seconds to find the optimal solution for the problem instance with no constraints (15S/10A) by the PSO algorithm, whereas it takes 11.70 seconds by the CPLEX solver.
- For the problem instances with only blocking time constraints, the average computational time by the PSO algorithm is less than that by the CPLEX solver. On the contrary, for the problem instances which have only a time window constraint, and those which have both blocking time and time window constraints, the average computational times by the PSO algorithm are higher than those by the CPLEX solver.
- By the CPLEX solver, the average computational time increases by only including the blocking time constraint in the problem instance, whereas the average computational time decreases by only including the time window constraint. This shows that blocking time constraints have much more impact than time window constraints on the complexity for the MILP model. However, we observe just the opposite for the PSO algorithm, which means that time window constraints affect the problem complexity more than the blocking time constraints.
- By the PSO algorithm, the highest deviation of 1.92 % and the highest computational time of 87.20 seconds are observed for problem instance (15S/10A/B/T-4) with both blocking time and time window constraints.

Table 5.3 MILP and PSO Solutions for 15S/10A with SP-Graph

<i>Graph Type</i>		<i>Problem Instance</i>	<i>Z_{MILP}</i>	<i>Optimality Gap</i>	<i>CPU (sec)</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>
SP		15S/10A	66835	0.00%	11.7	66835	13.88	0
		15S/10A/B-1	66835	0.00%	30.95	66835	15.81	0
		15S/10A/B-2	66835	0.00%	11.97	66835	15.37	0
		15S/10A/B-3	66410	0.00%	12.59	66410	14.04	0
		15S/10A/B-4	66835	0.00%	12.89	66835	12.85	0
		15S/10A/B-5	66410	0.00%	23.64	66500	12.83	0.14
		Average of 15S/10A/B		0.00%	18.41		14.18	0.03
		15S/10A/T-1	68600	0.00%	2.05	68940	55.86	0.5
		15S/10A/T-2	80645	0.00%	0.39	80790	21.09	0.18
		15S/10A/T-3	69690	0.00%	1.83	69785	48.51	0.14
		15S/10A/T-4	68270	0.00%	4.53	68520	33.94	0.37
		15S/10A/T-5	69070	0.00%	1.94	69180	38.73	0.16
		Average of 15S/10A/T		0.00%	2.14		39.63	0.27
		15S/10A/B/T-1	68940	0.00%	1.45	68940	55.37	0
		15S/10A/B/T-2	81740	0.00%	0.27	81745	28.51	0.01
		15S/10A/B/T-3	69615	0.00%	1.08	69680	51.46	0.09
		15S/10A/B/T-4	68270	0.00%	4.45	69580	87.2	1.92
		15S/10A/B/T-5	69315	0.00%	0.88	69885	37.32	0.82
		Average of 15S/10A/B/T		0.00%	1.63		51.97	0.57

- The average computational time and the average percent deviation are higher in the problem instances with only time window constraints than in the problem instances with only blocking time constraints by the PSO algorithm. In other words, the time window constraint affects the complexity of the problem much more than the blocking time constraint when using the PSO algorithm.
- The highest average computational time is observed with the problem instances, including both blocking time and time window constraints by the PSO algorithm. On the contrary, the lowest average computational time is observed with the problem instances, including both blocking time and time window constraints by the CPLEX solver.

- The optimal solutions are obtained in 6 out of 16 problem instances using the PSO algorithm.
- As blocking time and time window constraints are imposed in the problem instances, the average computational times increase by the PSO algorithm.

When we analyze Table 5.4 and Figure 5.4, we can make the following observations for the problem instances with 15 scenes and 10 actors with precedence relationships of the OF-graph type:

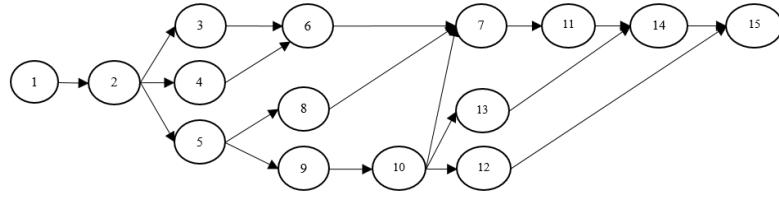


Figure 5.4. OF-Graph for 15S/10A

- The PSO algorithm solves problem instances with no constraints with an average deviation of 0.28 % in 16.42 seconds. However, the optimal solution is obtained in 2.81 seconds by the CPLEX solver.
- The average computational time by the CPLEX solver is higher in the problem instances with only blocking time constraints than in the problem instances with only time window constraints. This indicates that blocking time constraints have much more impact on the complexity of the MILP model than the time window constraints for the CPLEX solver.
- The average computational time and the average percent deviation are higher in the problem instances with only time window constraints than in the problem instances with only blocking time constraints by the PSO algorithm. In other words, the time window constraint affects the complexity of the problem much more than the blocking time constraint when using the PSO algorithm.
- Only one problem instance with both blocking time and time window constraints (15S/10A/B/T-2) is solved to optimality by the PSO algorithm.

- The highest computational time of 344.42 seconds is observed in problem instance (15S/10A/B/T-4) with both blocking time and time window constraints by the PSO algorithm.

Briefly, for the problem instances with 15 scenes/10 actors, it can be concluded that the highest average computational time is observed in problem instances with both blocking time and time window constraints in both graph types, meaning that it takes more time to find an optimal or near-optimal solution for the problems with all constraints by the PSO algorithm.

However, the opposite is observed for the MILP model solved by the CPLEX solver. The lowest average computational time by the CPLEX solver is observed as both blocking time and time window constraints are involved in the problem instances regardless of the precedence relations graph types. This means that as constraints are included in the problem instances, the solution space is reduced, and the average computational time decreases by the CPLEX solver.

For the problem instances with only blocking time constraints, the average computational time by the CPLEX solver increases regardless of the precedence relation graph types.

By the PSO algorithm, the average computational time increases with only blocking time constraints in the problem instances of the SP-graph type. However, the contrary is observed for the problem instances of the OF-graph type. Therefore, we can infer that problems with only blocking time constraints with the OF-graph type are solved in less computational time than those with the SP-graph type.

5.2.1.3 Problem Instances with 20 Scenes and 10 Actors

In this section, 20 scenes and 10 actors are involved in the test problems. SP-graph and OF-graph types are constructed to describe the precedence relations among the scenes.

Table 5.4 MILP and PSO Solutions for 15S/10A with OF-Graph

<i>Graph Type</i>	<i>Problem Instance</i>	Z_{MILP}	<i>Optimality Gap</i>	<i>CPU (sec)</i>	Z_{PSO}	<i>CPU (sec)</i>	<i>PD</i>
OF	15S/10A	67105	0.00%	2.81	67295	16.42	0.28
	15S/10A/B-1	67105	0.00%	3.16	67265	14.63	0.24
	15S/10A/B-2	67105	0.00%	3.11	67185	15.36	0.12
	15S/10A/B-3	67105	0.00%	2.83	67180	13.37	0.11
	15S/10A/B-4	67105	0.00%	3.23	67180	13.45	0.11
	15S/10A/B-5	66135	0.00%	3.59	67180	13.92	1.58
	Average of 15S/10A/B		0.00%	3.18		14.15	0.43
	15S/10A/T-1	68575	0.00%	0.25	68765	26.06	0.28
	15S/10A/T-2	77535	0.00%	0.05	78435	23.67	1.16
	15S/10A/T-3	68575	0.00%	0.34	68620	81.7	0.07
	15S/10A/T-4	67910	0.00%	1.33	68400	41.81	0.72
	15S/10A/T-5	67800	0.00%	0.7	67865	18.29	0.1
	Average of 15S/10A/T		0.00%	0.53		38.31	0.46
	15S/10A/B/T-1	68390	0.00%	0.17	68875	28.52	0.71
	15S/10A/B/T-2	88560	0.00%	0.03	88560	25.56	0
	15S/10A/B/T-3	68575	0.00%	0.42	68910	117.22	0.49
	15S/10A/B/T-4	68060	0.00%	0.3	68300	344.42	0.35
	15S/10A/B/T-5	70340	0.00%	0.52	70420	15.81	0.11
	Average of 15S/10A/B/T		0.00%	0.29		106.31	0.33

Figures 5.5 and 5.6 represent the SP-graph and OF-graph types, respectively. The solutions are provided in Tables 5.5 and 5.6 for these precedence relation graph types.

When we analyze Table 5.5 and Figure 5.5, we can make the following observations for the problem instances with 20 scenes and 10 actors with precedence relationships of the SP-graph type:

- Problem instances with no constraints are solved with an average deviation of 0.13 % in 18.62 seconds by the PSO algorithm. In contrast, the optimal solution is obtained in 9118.12 seconds by the CPLEX solver.

- Problem instances with only blocking time constraints are solved by the CPLEX solver by limiting the solution time to 3 hours (10800 seconds) because of the long computational times to optimality.

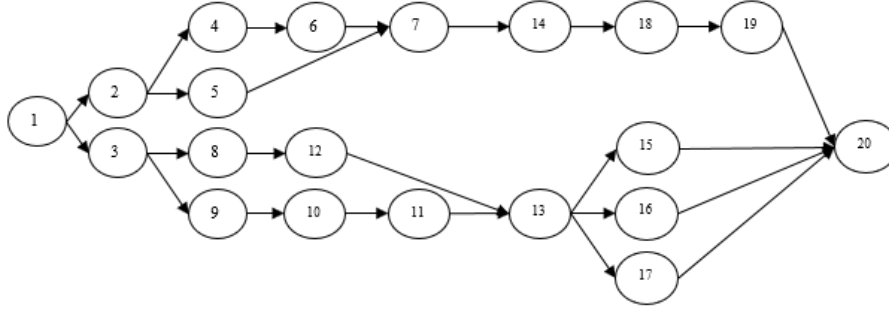


Figure 5.5. SP-Graph for 20S/10A

- By the CPLEX solver, when including only blocking time constraints in the problem, the average computational time does not decrease. In contrast, when imposing only time window constraints in the problem instances, the average computational time decreases. In addition, the shortest computational time is observed when imposing both blocking time and time window constraints. This means that including only the blocking time constraints in the problems does not reduce the solution space. However, imposing the time window constraints reduces the solution space and average computational time, as well.
- Only one problem instance with both blocking time and time window constraints (20S/10A/B/T-2) is solved to optimality by the PSO algorithm.
- By the PSO algorithm, the highest deviation of 1.90 % and the highest computational time of 115.25 seconds are observed with problem instances (20S/10A/B/T-4 and 20S/10A/B/T-5) with both blocking time and time window constraints, respectively.
- As blocking time and time window constraints are imposed in the problem instances, the average computational times increase by the PSO algorithm.
- For the problem instances with only time window constraints, it takes only 66.89 seconds to solve the problems with an average deviation of 0.84 % by

the PSO algorithm. On the other hand, the average computational time is 149.69 seconds by the CPLEX solver.

Table 5.5 MILP and PSO Solutions for 20S/10A with SP-Graph

<i>Graph Type</i>	<i>Problem Instance</i>	<i>Z_{MILP}</i>	<i>Optimality Gap</i>	<i>CPU (sec)</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>
SP	20S/10A	78960	0.00%	9118.12	79065	18.62	0.13
	20S/10A/B-1	78960	8.22%	10800	79260	23.35	0.38
	20S/10A/B-2	78960	3.05%	10800	79035	19.22	0.09
	20S/10A/B-3	78535	8.34%	10800	79080	18.97	0.69
	20S/10A/B-4	78645	8.53%	10800	79540	17.75	1.14
	20S/10A/B-5	78100	12.80%	10800	79260	17.34	1.49
	Average of 20S/10A/B		8.19%	10800		19.33	0.76
	20S/10A/T-1	79555	0.00%	179.05	80075	83.08	0.65
	20S/10A/T-2	89960	0.00%	10.88	90030	26.61	0.08
	20S/10A/T-3	80205	0.00%	139.44	80910	79.67	0.88
	20S/10A/T-4	80245	0.00%	309.33	81650	39.84	1.75
	20S/10A/T-5	80420	0.00%	109.77	81085	105.26	0.83
	Average of 20S/10A/T		0.00%	149.69		66.89	0.84
	20S/10A/B/T-1	79885	0.00%	102.25	80075	81.76	0.24
	20S/10A/B/T-2	90905	0.00%	11.02	90905	41.34	0
	20S/10A/B/T-3	80195	0.00%	34.11	80745	103.01	0.69
	20S/10A/B/T-4	79860	0.00%	67.89	81380	89.69	1.9
	20S/10A/B/T-5	79800	0.00%	37.13	81065	115.25	1.59
	Average of 20S/10A/B/T		0.00%	50.48		86.21	0.88

- The average computational time and the average percent deviation by the PSO algorithm are higher in problem instances with only time window constraints than in problem instances with only blocking time constraints. In other words, the time window constraint affects the complexity of the problem much more than the blocking time constraint when using the PSO algorithm.
- For the problem instances, including both blocking time and time window constraints, the average computational time is 50.48 seconds by the CPLEX

solver. On the contrary, by the PSO algorithm, the problem instances with both blocking time and time window constraints are solved with an average deviation of 0.88% in about 86.21 seconds.

When we analyze Table 5.6 and Figure 5.6, we can make the following observations for the problem instances with 20 scenes and 10 actors with precedence relationships of the OF-graph type:

- For the problem instance with no constraints, the optimal solution is found in 257.69 seconds by the CPLEX solver. But, by the PSO algorithm, a solution with a deviation of 0.71 % is observed in 25.26 seconds.
- The highest average deviation of 1.03 % is observed for problem instances that include only time window constraints. However, the lowest average deviation of 0.58 % is observed for problem instances that have both blocking time and time window constraints by the PSO algorithm.
- As blocking time and time window constraints are imposed in the problem instances, the average computational times decrease by the CPLEX solver.
- None of the problem instances can be solved to optimality by the PSO algorithm.
- The highest average computational time of 96.06 seconds is observed for the problem instance including both blocking time and time window constraints by the PSO algorithm. However, the largest average computational time is 257.69 seconds for the problem instance with no constraints by the CPLEX solver.

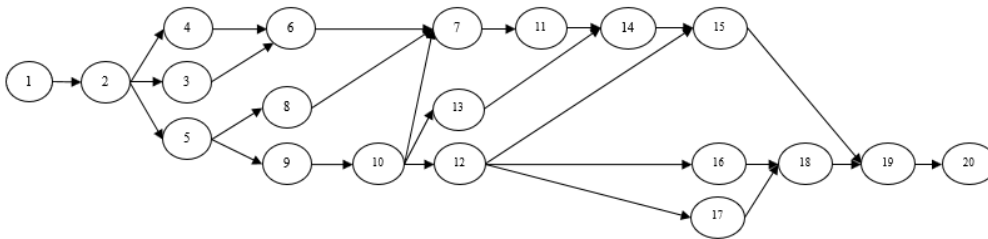


Figure 5.6. OF-Graph for 20S/10A

Table 5.6 MILP and PSO Solutions for 20S/10A with OF-Graph

<i>Graph Type</i>	<i>Problem Instance</i>	Z_{MILP}	<i>Optimality Gap</i>	<i>CPU (sec)</i>	Z_{PSO}	<i>CPU (sec)</i>	<i>PD</i>
OF	20S/10A	77825	0.00%	257.69	78375	25.26	0.71
	20S/10A/B-1	77825	0.00%	214.98	78350	20.69	0.67
	20S/10A/B-2	77825	0.00%	226.25	78380	17.51	0.71
	20S/10A/B-3	77650	0.00%	226	78355	17.39	0.91
	20S/10A/B-4	77825	0.00%	238.61	78060	17.49	0.3
	20S/10A/B-5	77055	0.00%	261.56	77495	17.54	0.57
	Average of 20S/10A/B		0.00%	233.48		18.12	0.63
	20S/10A/T-1	79930	0.00%	4.73	80485	36.33	0.69
	20S/10A/T-2	87415	0.00%	4.98	88425	32.65	1.16
	20S/10A/T-3	79925	0.00%	1.7	80220	116.33	0.37
	20S/10A/T-4	78660	0.00%	15.73	79470	70.84	1.03
	20S/10A/T-5	79855	0.00%	9.17	81360	44.63	1.88
	Average of 20S/10A/T		0.00%	7.26		60.16	1.03
	20S/10A/B/T-1	81000	0.00%	1.66	81170	42.05	0.21
	20S/10A/B/T-2	100105	0.00%	0.52	100335	35.3	0.23
	20S/10A/B/T-3	79750	0.00%	1.5	80045	164.99	0.37
	20S/10A/B/T-4	79760	0.00%	2.03	80155	184.66	0.57
	20S/10A/B/T-5	81350	0.00%	8.94	82590	53.3	1.52
	Average of 20S/10A/B/T		0.00%	2.93		96.06	0.58

- When imposing only blocking time constraints in problem instances, the average computational time decreases by the PSO algorithm. In contrast, it increases by imposing only the time window constraints in problem instances.

Consequently, for the problem instances with 20 scenes/10 actors, when using the PSO algorithm it can be inferred that the highest average computational time is observed as both blocking time and time window constraints are involved in the problem instances regardless of the precedence relations graph types.

As for the CPLEX solver, the lowest average computational time is observed as both blocking time and time window constraints are involved in the problem instances regardless of the precedence relations graph types. This indicates that as

constraints are included in the problem instances, the solution space is reduced, and as a result, the average computational time decreases.

5.2.1.4 Problem Instances with 25 Scenes and 10 Actors

In this section, 25 scenes and 10 actors are involved in the test problems with both SP-graph and OF-graph types. In Figures 5.7 and 5.8, the SP-graph and OF-graph types are given, and the solutions are provided in Tables 5.7 and 5.8, respectively.

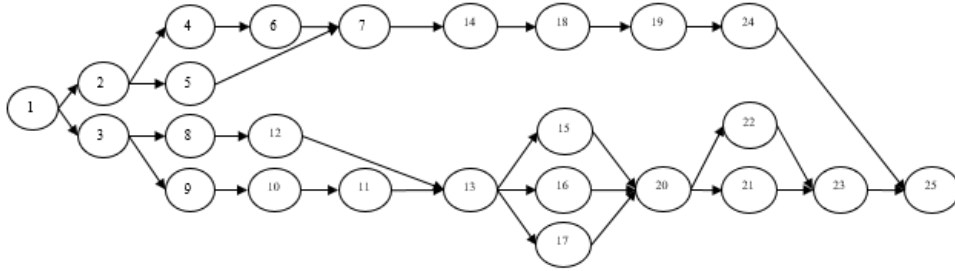


Figure 5.7 SP-Graph for 25S/10A

When we analyze Table 5.7 and Figure 5.7, we can make the following observations for the problem instances with 25 scenes and 10 actors with precedence relationships of the SP-graph type:

- Problem instances both without constraints and with only blocking time constraints are solved by the CPLEX solver by limiting the solution time to 3 hours (10800 seconds) because of the long computational times.
- By the CPLEX solver, as including only blocking time constraints in the problem, the average time required to solve the problems does not decrease, compared to the without-constraints instance (25S/10A). However, when imposing only the time window constraints in problem instances, the average computational time decreases.
- By the CPLEX solver, the lowest average computational time of 1179.06 seconds is observed when imposing both blocking time and time window constraints.

- When imposing only the time window constraints, the solution space is reduced, and as a result, the problem instances are solved relatively in a shorter time by the CPLEX solver.
- By the PSO algorithm, among 16 problem instances, only one optimal solution is obtained for the problem instance (25S/10A/B/T-2), which has both blocking time and time window constraints.
- By the PSO algorithm, the average computational times increase as blocking time and time window constraints are involved in the problems.

Table 5.7 MILP and PSO Solutions for 25S/10A with SP-Graph

<i>Graph Type</i>	<i>Problem Instance</i>	<i>Z_{MILP}</i>	<i>Optimality Gap</i>	<i>CPU (sec)</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>
SP	25S/10A	102500	25.14%	10800	102765	22.15	0.26
	25S/10A/B-1	102500	25.60%	10800	102795	23.53	0.29
	25S/10A/B-2	102500	23.98%	10800	102895	24.03	0.39
	25S/10A/B-3	102075	26.38%	10800	102340	20.93	0.26
	25S/10A/B-4	102185	26.53%	10800	102800	21.08	0.6
	25S/10A/B-5	101775	26.64%	10800	103090	21.48	1.29
	Average of 25S/10A/B		25.18%	10800		22.21	0.57
	25S/10A/T-1	103225	0.00%	10274	103955	110.59	0.71
	25S/10A/T-2	112055	0.00%	9.67	112205	35.85	0.13
	25S/10A/T-3	103525	0.00%	1487.83	103695	140.21	0.16
	25S/10A/T-4	103565	0.00%	4934.09	104055	155.29	0.47
	25S/10A/T-5	103870	0.00%	2913.83	104145	162.63	0.26
	Average of 25S/10A/T		0.00%	3923.88		120.91	0.35
	25S/10A/B/T-1	103560	0.00%	3853.22	104035	104.49	0.46
	25S/10A/B/T-2	113060	0.00%	10.45	113065	56.98	0
	25S/10A/B/T-3	103520	0.00%	608.97	103840	193.26	0.27
	25S/10A/B/T-4	103180	0.00%	621.73	103910	113.78	0.71
	25S/10A/B/T-5	103315	0.00%	800.92	105945	145.71	2.55
	Average of 25S/10A/B/T		0.00%	1179.06		122.84	0.8

- The highest computational time of 193.26 seconds by the PSO algorithm is observed for the problem instance (25S/10A/B/T-3) that has both blocking time and time window constraints.
- The lowest average deviation of 0.26% is observed for the problem instance with no constraints (25S/10A) in 22.15 seconds by the PSO algorithm.

When we analyze Table 5.8 and Figure 5.8, we can make the following observations for the problem instances with 25 scenes and 10 actors with precedence relationships of the OF-graph type:

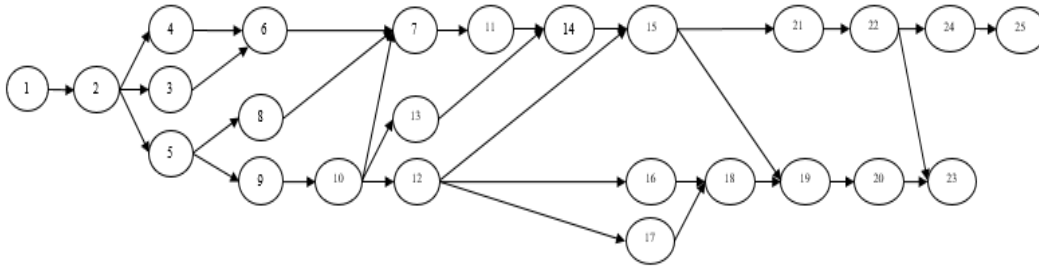


Figure 5.8 OF-Graph for 25S/10A

- Problem instances without constraints and only with blocking time constraints are solved by the CPLEX solver by limiting the solution time to 3 hours (10800 seconds) because of their long computational times.
- By the CPLEX solver, when including only blocking time constraints in the problem, the average computational time does not decrease compared to the without-constraint instance. In contrast, when imposing only the time window constraints in problem instances, the average computational time decreases. In addition, the lowest computational time is observed when imposing both blocking time and time window constraints. This means that including the blocking time constraints in the problems does not reduce the solution space. However, imposing the time window constraints reduces both the solution space and average computational time.
- As for the PSO algorithm, none of the problem instances is solved to optimality.

- By the PSO algorithm, when only blocking time constraints are included in the problems, the average computational time decreases, while it increases when only the time window constraints or both blocking and time window constraints are included. This shows that the time window constraints increase the complexity of the problem. But the highest average computational time is observed in problem instances with both blocking time and time window constraints.
- However, the highest average percent deviation is observed in problem instances with only time window constraints by the PSO algorithm.

Table 5.8 MILP and PSO Solutions for 25S/10A with OF-Graph

Graph Type	Problem Instance	Z_{MILP}	Optimality Gap	CPU (sec)	Z_{PSO}	CPU (sec)	PD
OF	25S/10A	102035	5.89%	10800	102530	23.94	0.49
	25S/10A/B-1	101835	6.99%	10800	102475	24.2	0.63
	25S/10A/B-2	102035	5.78%	10800	102240	23.35	0.2
	25S/10A/B-3	101860	9.38%	10800	102495	21.42	0.62
	25S/10A/B-4	102035	9.15%	10800	102230	21.97	0.19
	25S/10A/B-5	101065	8.79%	10800	101595	21.91	0.52
	Average of 25S/10A/B		8.02%	10800		22.57	0.43
	25S/10A/T-1	104580	0.00%	150.64	105430	40.61	0.81
	25S/10A/T-2	114565	0.00%	10.36	115725	42.51	1.01
	25S/10A/T-3	103185	0.00%	94.48	103490	214.38	0.3
	25S/10A/T-4	102335	0.00%	492.14	103045	115.96	0.69
	25S/10A/T-5	103105	0.00%	77.38	105060	272.2	1.9
	Average of 25S/10A/T		0.00%	165		137.13	0.94
	25S/10A/B/T-1	104780	0.00%	15.38	105300	51.41	0.5
	25S/10A/B/T-2	123895	0.00%	7.55	123965	55.56	0.06
	25S/10A/B/T-3	103010	0.00%	36.75	103340	188.78	0.32
	25S/10A/B/T-4	103375	0.00%	34.16	104760	174.65	1.34
	25S/10A/B/T-5	104400	0.00%	24.03	104930	227.9	0.51
	Average of 25S/10A/B/T		0.00%	23.57		139.66	0.54

To sum up, for the problem instances with 25 scenes/10 actors, it can be inferred that the highest computational time is observed for the problem instances with both

blocking time and time window constraints in both graph types by the PSO algorithm. The highest average percent deviation is observed for the problems that have only time window constraints with the OF-graph type by the PSO algorithm. However, the highest average percent deviation is observed for the problems that have both blocking time and time window constraints with the SP-graph type.

When imposing the time window constraints in the problem instances, the solution space is reduced, and as a result, the average computational times in both graph types decrease by the CPLEX solver. All problem instances with the OF-graph type, except the ones with only blocking time constraints, are solved in a shorter time than those with the SP-graph type by the CPLEX solver.

5.2.1.5 Problem Instances with 25 Scenes and 15 Actors

In this section, the test problems involve 25 scenes and 15 actors. Firstly, the SP-graph in Figure 5.7 is used for the precedence relations among scenes, and the corresponding solutions are provided in Table 5.9. Secondly, the OF-graph in Figure 5.8 is given, and the related solutions are introduced in Table 5.10.

When we analyze Table 5.9, we can make the following observations for the problem instances with 25 scenes and 15 actors with precedence relationships of the SP-graph type:

- Problem instances with no constraints and with only blocking time constraints are both solved by the CPLEX solver by limiting the solution time to 3 hours (10800 seconds) because of their long computational times.
- By the CPLEX solver, when including only the blocking time constraints in the problem, the average computational time does not decrease. However, when imposing only time window constraints in the problem instances, the average computational time decreases. The lowest average computational time of 1041.29 seconds is observed when imposing both blocking time and time window constraints.

- Only one optimal solution among 16 problem instances is obtained for the problem instance (25S/15A/B/T-2), which has both blocking time and time window constraints by the PSO algorithm.
- The lowest deviation of 0.15 % is observed for the problem instance with no constraints (25S/15A), while the highest average deviation of 0.67 % is observed for the problem instances with both blocking time and time window constraints by the PSO algorithm.
- By the PSO algorithm, as the constraints are imposed in the problem instances, the average solution times increase, and the highest computational time, that is 246.44 seconds, is observed for the problem instance (25S/15A/B/T-4) with both blocking time and time window constraints.

Table 5.9 MILP and PSO Solutions for 25S/15A with SP-Graph

<i>Graph Type</i>	<i>Problem Instance</i>	<i>Z_{MILP}</i>	<i>Optimality Gap</i>	<i>CPU (sec)</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>
SP	25S/15A	116370	23.09%	10800	116540	24.58	0.15
	25S/15A/B-1	116095	24.23%	10800	116465	24.52	0.32
	25S/15A/B-2	116370	24.11%	10800	116620	29.87	0.21
	25S/15A/B-3	115705	27.24%	10800	116580	25.54	0.76
	25S/15A/B-4	115440	26.28%	10800	116595	24.04	1
	25S/15A/B-5	115435	25.53%	10800	116355	26.71	0.8
	Average of 25S/15A/B		25.48%	10800		26.14	0.62
	25S/15A/T-1	116595	3.02%	10800	117290	96.61	0.6
	25S/15A/T-2	124870	0.00%	110.56	125005	44.19	0.11
	25S/15A/T-3	116490	0.00%	2687.66	117360	103.39	0.75
	25S/15A/T-4	116490	0.00%	5311.56	116890	113.33	0.34
	25S/15A/T-5	116850	0.00%	2380.81	117145	149.99	0.25
	Average of 25S/15A/T		0.60%	4258.12		101.5	0.41
	25S/15A/B/T-1	117455	0.00%	3262.81	119000	115.09	1.32
	25S/15A/B/T-2	130610	0.00%	86.42	130615	65.62	0
	25S/15A/B/T-3	116315	0.00%	691.88	116895	219.22	0.5
	25S/15A/B/T-4	115555	0.00%	92.61	116050	246.44	0.43
	25S/15A/B/T-5	117235	0.00%	1072.73	118550	145.8	1.12
	Average of 25S/15A/B/T		0.00%	1041.29		158.43	0.67

When we analyze Table 5.10, we can make the following observations for the problem instances with 25 scenes and 15 actors with precedence relationships of the OF-graph type:

- Problem instances without constraints and with only blocking time constraints are solved by the CPLEX solver by limiting the solution time to 3 hours (10800 seconds) because of their long computational time.
- By the CPLEX solver, when including only blocking time constraints in the problem, the average computational time does not decrease. However, when imposing only the time window constraints in the problem instances, the average computational time decreases.

Table 5.10 MILP and PSO Solutions for 25S/15A with OF-Graph

<i>Graph Type</i>	<i>Problem Instance</i>	<i>Z_{MILP}</i>	<i>Optimality Gap</i>	<i>CPU (sec)</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>
OF	25S/15A	113550	5.72%	10800	113695	30.41	0.13
	25S/15A/B-1	113475	6.92%	10800	113720	26.68	0.22
	25S/15A/B-2	113550	6.32%	10800	114250	23.83	0.62
	25S/15A/B-3	113375	7.70%	10800	113985	25.09	0.54
	25S/15A/B-4	113550	7.51%	10800	114235	26.92	0.6
	25S/15A/B-5	112580	7.92%	10800	113015	23.85	0.39
	Average of 25S/15A/B		7.72%	10800		25.27	0.47
	25S/15A/T-1	117780	0.00%	217.21	118525	43.38	0.63
	25S/15A/T-2	127640	0.00%	12.38	134335	42.81	5.25
	25S/15A/T-3	115860	0.00%	100.25	116405	204.13	0.47
	25S/15A/T-4	114085	0.00%	582.08	114350	125.79	0.23
	25S/15A/T-5	114615	0.00%	60.7	118080	221.3	3.02
	Average of 25S/15A/T		0.00%	194.52		127.48	1.92
	25S/15A/B/T-1	117735	0.00%	18.95	119305	44.59	1.33
	25S/15A/B/T-2	140700	0.00%	7.58	141985	47.3	0.91
	25S/15A/B/T-3	115685	0.00%	80.44	115990	237.84	0.26
	25S/15A/B/T-4	115750	0.00%	36.27	115945	41.54	0.17
	25S/15A/B/T-5	115970	0.00%	18.92	116400	388.91	0.37
	Average of 25S/15A/B/T		0.00%	32.43		152.04	0.61

- By the CPLEX solver, the lowest average computational time of 32.43 seconds is observed when imposing both blocking time and time window constraints.
- As for the PSO algorithm, the highest computational time of 388.91 seconds is observed for the problem instance (25S/15A/B/T-5), which has both blocking time and time window constraints.
- When imposing only the blocking time constraints in the problem instances, the average computational time decreases. However, the average computational time increases when imposing only the time window constraints in the problem instances. This shows that the time window constraints are more challenging for the PSO algorithm.
- The lowest average deviation of 0.13 % is observed for the problem instance with no constraints by the PSO algorithm.

For the problem instances with 25 scenes/15 actors, it can be inferred that the highest computational time is observed for the problem instance with both blocking time and time window constraints with both graph types by the PSO algorithm. The highest average percent deviation is observed for the problems that have only time window constraints with the OF-graph type by the PSO algorithm. However, the highest average percent deviation is observed for the problems that have both blocking time and time window constraints with the SP-graph type.

By the CPLEX solver, when imposing the time window constraints in the problem instances, the solution space is reduced, and as a result, the average computational times in both graph types decrease. All problem instances with the OF-graph type, except the ones with only blocking time constraints, are solved in a shorter time than those with the SP-graph type.

5.2.2 A Real Film Script Instance

In this section, a problem instance sourced completely from a real film script by Şahin (2007) is used to examine the performance of the PSO algorithm for a real-life size problem. In this film script, there are 83 scenes to be shot, and 52 actors are required for the whole film's scenes. Precedence relations among the scenes have been generated according to the scene characteristics in a logical framework. In the precedence relations graph, additionally four dummy scenes are defined: one is for the start scene as scene 0, and the other is for the end scene as scene 92 so that the MILP model and precedence relations would be defined correctly. Moreover, two additional dummy nodes are used: the first is used for linking the night and day scenes at the same shooting site, and the second is used for connecting the scenes of 'Gürol' character regarding his old age and youth. The precedence relations among the scenes are given in Appendices G, H, and I.

We introduce three preprocessing rules to reduce the feasible region of the problem. As a result, the efficiency of the solution methodology is boosted. As the first rule, the scenes in which the same actors participate and are to be shot at the same shooting sets are merged. To give an example for this rule, scenes 61 and 63 are combined because the same actors play at the same shooting set. As the second rule; the scenes that are shot at the shooting sets close to each other and tell a sequence of events are merged as one scene. For instance, scenes 29 and 30 tell a story, and they are shot in the garden of 'Müjde's house and the street. As the third rule; only the actors who have conversations are considered as the main actors, while the others are considered as figurants and therefore eliminated from the formulation of the problem. For instance, in scene 57, 'Emin' does not have any conversation, so 'Emin' is assumed as a figurant in the shooting, and thus eliminated. The merged scenes and the corresponding scene numbers in Şahin (2007) are paired as in Appendices J and K. The scene-actor assignment matrix is prepared according to the roles in the scenes (Appendix L). Finally, this real film script is formulated as an MILP model with its 70 scenes and 43 actors and then

solved by the GAMS version 23.9.5 on the PC with 16 GB RAM, Intel ® Xeon ® E-2246G @ 3.60 GHz, Windows 10 version. The MILP model is run continuously for 15 days, but the optimum solution cannot be obtained due to the large size of the problem. The best integer solution turned out to have an objective function value of 190,740 with an optimality gap of 88.87 % by the end of 5.6 days. Therefore, it can be concluded that the problem cannot be solved due to its large size, and as a result, in order to obtain a better solution, the problem is solved by the PSO algorithm with different numbers of particles and iterations as discussed in Section 5.3.2 below.

5.3 Effects of the Numbers of Particles and Iterations on the PSO Performance

It is known that population size affects the performance of population-based metaheuristics significantly. In a PSO algorithm, individuals called as particles, which represent potential solutions to a problem evolve according to their own and companions' experiences through iterations. Therefore, as also mentioned in Section 4.1, the performance of a PSO is closely related to the selected number of particles and iterations. In this section, tests are performed on the generated problem instances and real film script instance with different numbers of particles and iterations using the PC with 8 GB RAM, 11th Gen Intel® Core™ i5-1135G7 @ 2.40 GHz, Windows 11 version.

5.3.1 Evaluation over the Generated Problem Instances

The numbers of actors and scenes determine the problem size. As a result, as discussed in Section 5.2.1, 160 problem instances of different sizes are solved to evaluate the PSO algorithm's performance. As provided in Figure 5.9 below, out of 160 problem instances, 30 problem instances are solved to optimality; 106 problem instances are solved with a deviation of less than 1.00 %; and only 24 problem

instances are solved with a deviation higher than 1.00 % and less than 2.00 % on the average by the PSO algorithm.

Deviations higher than 1.00 % may result due to the selected numbers of particles and iterations. To evaluate the effects of the numbers of particles and iterations, the smallest and the largest problem instances, 10S/10A/B-5 and 25S/15A/T-2 with OF-graph types, are solved once with a different number of particles and iterations. The related test results are provided in Tables 5.11-5.14.

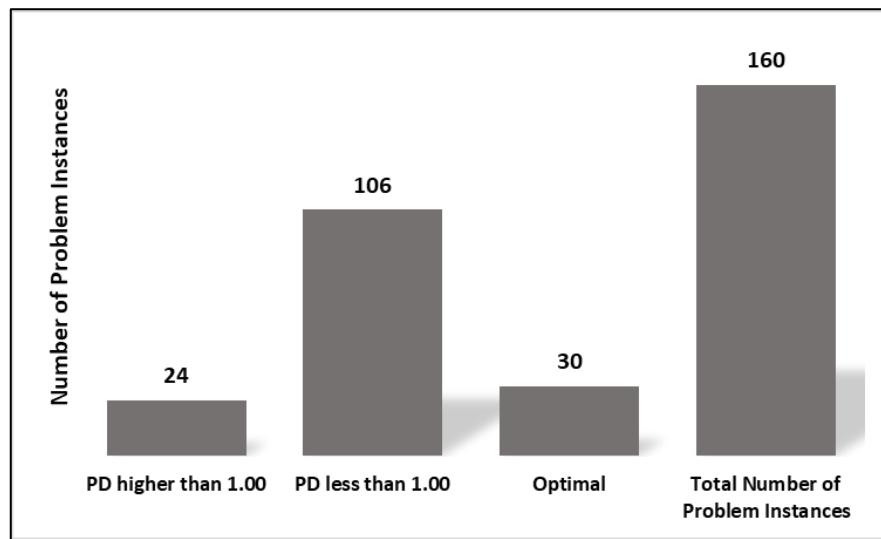


Figure 5.9 Quality of Solutions based on Percent Deviation (PD)

For problem instance 10S/10A/B-5 of the OF-graph type, the deviations lower/higher than 1.94 % may be obtained with a different number of particles and iterations. As provided in Table 5.11, we can reduce the deviation from 1.94 % to 1.06 % with the objective value of 51,170 with 70 particles for 40 and 50 iterations.

Similarly, for problem instance 25S/15A/T-2 of the OF-graph type, deviations lower/higher than 5.25 % may be obtained with a different number of particles and iterations as provided in Table 5.13. In addition, the deviation can be decreased to 1.09 % from 5.25 % with the objective value of 129,025 with 70 particles for 10 iterations. As a result, it can be inferred that the solution quality may be improved by increasing the number of particles and/or iterations.

We can also evaluate the PSO algorithm's performance with the help of increasing the number of iterations gradually by 10 until it reaches 60. The test results for problem instances 10S/10A/B-5 and 25S/15A/T-2 of OF-graph types are given in Tables 5.12 and 5.14, respectively.

In problem instance 10S/10A/B-5 of OF-graph type, the best objective value is obtained as 52,220 at the end of 10 iterations with 10 particles. The first improvement and the best objective value of 52,100 are observed at the end of 40 iterations. With 30 particles, the best objective value is obtained as 52,200 at the end of 10 iterations. The first improvement and the best objective value of 52,120 are observed at the end of 30 iterations. With 50 and 70 particles, the best objective values of 52,100 and 51,945 are obtained at the end of 10 iterations, respectively. However, no improvement is observed until the end of 60 iterations. With 90 particles, the best objective value is obtained as 52,200 at the end of 10 iterations. The first improvement and the best objective value of 52,100 are observed at the end of 20 iterations.

In problem instance 25S/15A/B-5 of OF-graph type, the best objective value is obtained as 143,415 at the end of 10 iterations with 10 particles. The first improvement and the best objective value of 133,715 are observed at the end of 30 iterations. With 30 particles, the best objective value is obtained as 137,600 at the end of 10 iterations. The first improvement and the best objective value of 132,220 are observed at the end of 20 iterations. With 50 particles, the best objective value is obtained as 134,605 at the end of 10 iterations. The first and second improvements are observed at the end of iterations 20 and 30 by the objective values of 132,220 and 131,780, respectively. With 70 particles, the best objective value of 132,950 is obtained at the end of 10 iterations. However, no improvement is observed until the end of 60 iterations. With 90 particles, the first and second improvements are observed at the end of 20 and 50 iterations by the objective values of 132,870 and 132,530, respectively.

It can also be observed on Figure 5.10 that increasing the number of particles rather than the number of iterations increases the computation time for both problem instances 10S/10A/B-5 and 25S/15A/T-2.

5.3.2 Evaluation over the Real Film Script

In this section, tests are performed on the problem instance of a real film script with different numbers of particles and iterations. The PSO algorithm is run 20 times for 10, 30, 50, 70, and 90 particles with iterations in the range of 10-60. Then minimum, average, and maximum values of the objective function values are provided in Table 5.15.

When we analyze Table 5.15, we observe that most solutions obtained by the PSO algorithm applied with different combinations of particles and iterations are better than the objective function value, 190,740, which is the best integer solution obtained by the CPLEX solver. The minimum objective value among all runs is obtained as 165,825 with 70 particles and 20 iterations.

The best two average objective values are obtained with 50 iterations-30 and 50 particles; the best three average objective values are obtained with 60 iterations-10, 70, and 90 particles.

The minimum objective function values among the minimum ones are observed with 40 iterations-10 and 30 particles, 50 iterations-50 particles, 20 iterations-70 particles, and 30 iterations-90 particles. In addition, it can be observed that for 10, 20, and 30 particles, the minimum objective values can be obtained after 30 iterations. However, for particles higher than 50, the minimum objective values can be found with 20 and 30 iterations.

In Figure 5.11, the graphs show the average objective function values for each number of particles and iterations. When we analyze Figure 5.11 (a) for a specific number of iterations, we can see the effect of the number of particles on the solution quality regarding the objective value.

Table 5.11 Effects of Numbers of Particles and Iterations – Problem Instance: 10S/10A/B-5 with OF-Graph

		<i>Number of Iterations</i>																	
		10			20			30			40			50			60		
		<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>
<i>Number of Particles</i>	10	52245	3.18	2.18	52120	4.17	1.94	52120	5.44	1.94	52120	7.04	1.94	52100	9.43	1.90	52100	10.91	1.90
	30	52100	5.21	1.90	52100	9.59	1.90	52045	15.72	1.79	51810	20.02	1.33	52200	23.35	2.09	52120	29.38	1.94
	50	51945	8.61	1.59	51810	14.27	1.33	51945	21.78	1.59	52100	28.24	1.90	52100	37.18	1.90	52100	42.12	1.90
	70	52120	11.68	1.94	52100	19.82	1.90	52100	29.67	1.90	51670	39.27	1.06	51670	48.86	1.06	51945	58.31	1.59
	90	51810	12.44	1.33	52120	23.68	1.94	51810	35.06	1.33	52045	46.93	1.79	51770	56.74	1.25	52100	70.46	1.90

Table 5.12 Effects of Numbers of Particles and Iterations in Sequence – Problem Instance: 10S/10A/B-5 with OF-Graph

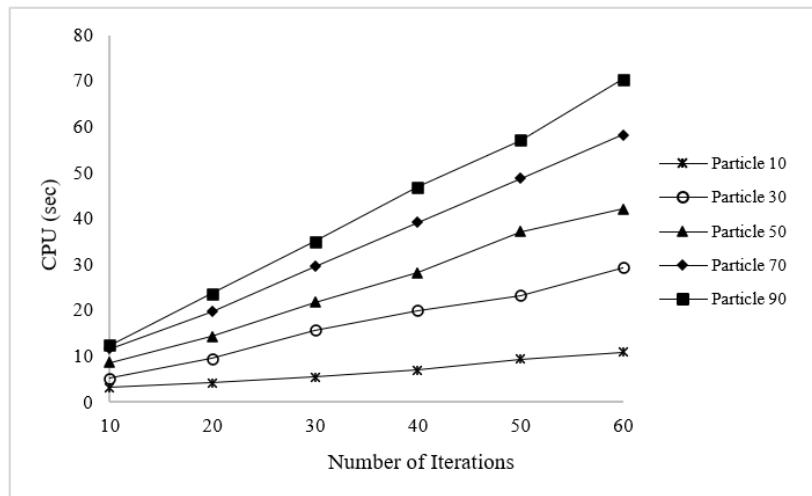
		<i>Number of Iterations</i>											
		10		20		30		40		50		60	
		<i>Z_{PSO}</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>PD</i>
<i>Number of Particles</i>	10	52220	2.13	52220	2.13	52220	2.13	52100	1.90	52100	1.90	52100	1.90
	30	52200	2.09	52200	2.09	52120	1.94	52120	1.94	52120	1.94	52120	1.94
	50	52100	1.90	52100	1.90	52100	1.90	52100	1.90	52100	1.90	52100	1.90
	70	51945	1.59	51945	1.59	51945	1.59	51945	1.59	51945	1.59	51945	1.59
	90	52200	2.09	52100	1.90	52100	1.90	52100	1.90	52100	1.90	52100	1.90

Table 5.13 Effects of Numbers of Particles and Iterations – Problem Instance: 25S/15A/T-2 with OF-Graph

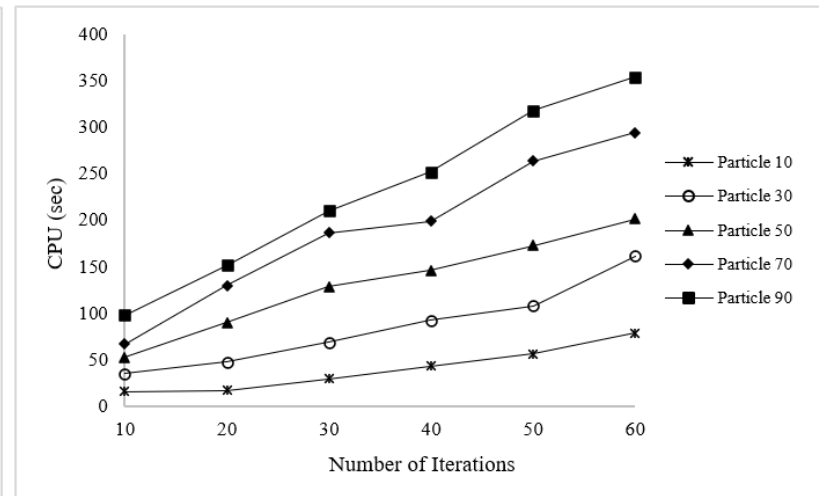
		<i>Number of Iterations</i>																	
		10			20			30			40			50			60		
		<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>PD</i>
<i>Number of Particles</i>	10	133640	15.42	4.70	134405	17.09	5.30	133005	29.57	4.20	132530	42.76	3.83	135855	36.29	6.44	133715	48.18	4.76
	30	133060	35.21	4.25	134100	47.29	5.06	130220	68.81	2.02	132900	92.27	4.12	134055	107.84	5.03	132220	161.45	3.59
	50	134165	52.26	5.11	132290	89.52	3.64	133745	128.45	4.78	132555	145.58	3.85	131440	172.92	2.98	131780	201.21	3.24
	70	129025	66.75	1.09	133835	129.81	4.85	131895	186.63	3.33	132275	198.57	3.63	133060	263.58	4.25	132950	293.89	4.16
	90	133640	97.93	4.70	133640	151.24	4.70	132290	249.51	3.64	129115	251.84	1.16	129720	317.62	1.63	132530	353.67	3.83

Table 5.14 Effects of Numbers of Particles and Iterations in Sequence– Problem Instance: 25S/15A/T-2 with OF-Graph

		<i>Number of Iterations</i>											
		10		20		30		40		50		60	
		<i>Z_{PSO}</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>PD</i>	<i>Z_{PSO}</i>	<i>PD</i>
<i>Number of Particles</i>	10	143415	12.36	143415	12.36	133715	4.76	133715	4.76	133715	4.76	133715	4.76
	30	137600	7.80	132220	3.59	132220	3.59	132220	3.59	132220	3.59	132220	3.59
	50	134605	5.46	132220	3.59	131780	3.24	131780	3.24	131780	3.24	131780	3.24
	70	132950	4.16	132950	4.16	132950	4.16	132950	4.16	132950	4.16	132950	4.16
	90	133835	4.85	132870	4.10	132870	4.10	132870	4.10	132530	3.83	132530	3.83



(a) Problem Instance: 10S/10A/B-5 with OF-Graph



(b) Problem Instance: 25S/15A/T-2 with OF-Graph

Figure 5.10 Effects of Numbers of Particles and Iterations on Computational Time

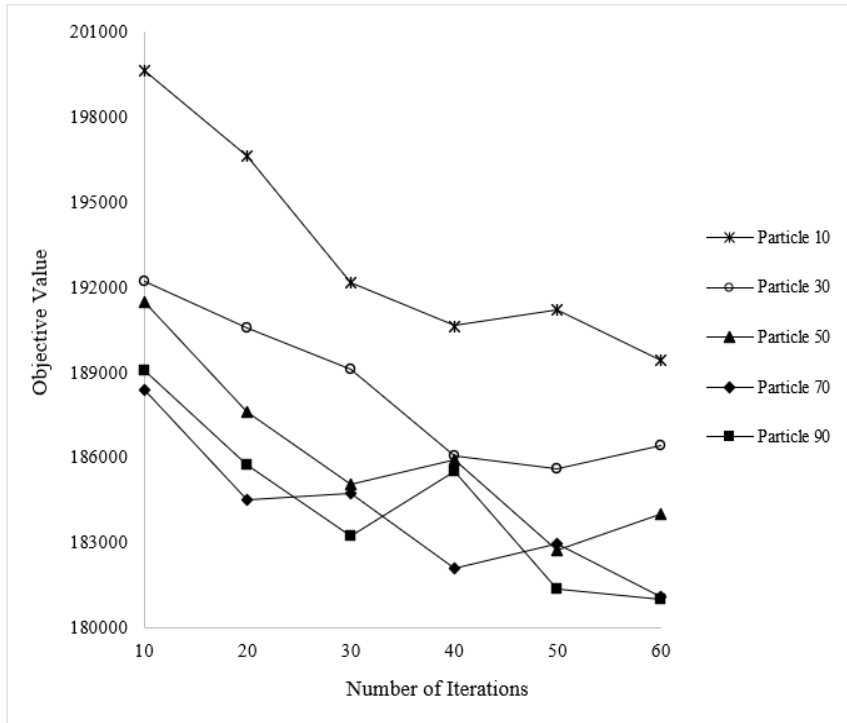
Table 5.15 Effects of Numbers of Particles and Iterations – Problem Instance: Real Film Script

			<i>Number of Iterations</i>											
			10		20		30		40		50		60	
			<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>	<i>Z_{PSO}</i>	<i>CPU (sec)</i>
<i>Number of Particles</i>	10	Min	190450	18.58	187325	34.72	181765	54.18	171110	33.28	179625	81.37	182955	98.04
		Avg	199631	19.69	196599	36.77	192153	59.46	190622	67.90	191206	85.08	189424	100.34
		Max	207205	22.61	205550	44.03	200510	74.38	199770	79.65	198130	89.55	195330	106.21
	30	Min	179285	53.66	184220	107.00	181765	141.84	177380	192.37	178975	241.12	182660	286.66
		Avg	192200	55.11	190543	111.02	189074	145.22	186026	199.55	185596	255.52	186423	294.22
		Max	201445	61.97	198435	127.46	195810	156.28	193555	215.49	192200	286.91	193570	318.71
	50	Min	179345	90.15	176365	172.61	173520	232.33	177695	361.14	170680	384.76	177390	474.93
		Avg	191445	101.27	187576	177.57	185036	244.99	185904	382.26	182705	395.46	183998	493.35
		Max	196960	112.29	193510	184.05	193275	340.53	191180	405.66	191325	401.98	188910	515.64
	70	Min	177500	177.68	165825	233.38	175605	338.57	176385	462.97	175365	552.17	168500	671.11
		Avg	188367	207.53	184479	238.36	184703	348.00	182066	473.33	182919	585.43	181072	679.68
		Max	196805	256.52	192430	255.48	189745	421.98	189250	481.81	190270	649.18	185900	687.92
	90	Min	180920	206.43	180080	304.01	172410	444.19	178810	586.28	174455	715.08	174285	895.69
		Avg	189042	300.23	185739	314.05	183208	451.65	185474	596.79	181362	760.47	181001	917.92
		Max	195190	339.65	191465	347.82	189720	479.68	189890	617.47	189760	778.36	186745	936.98

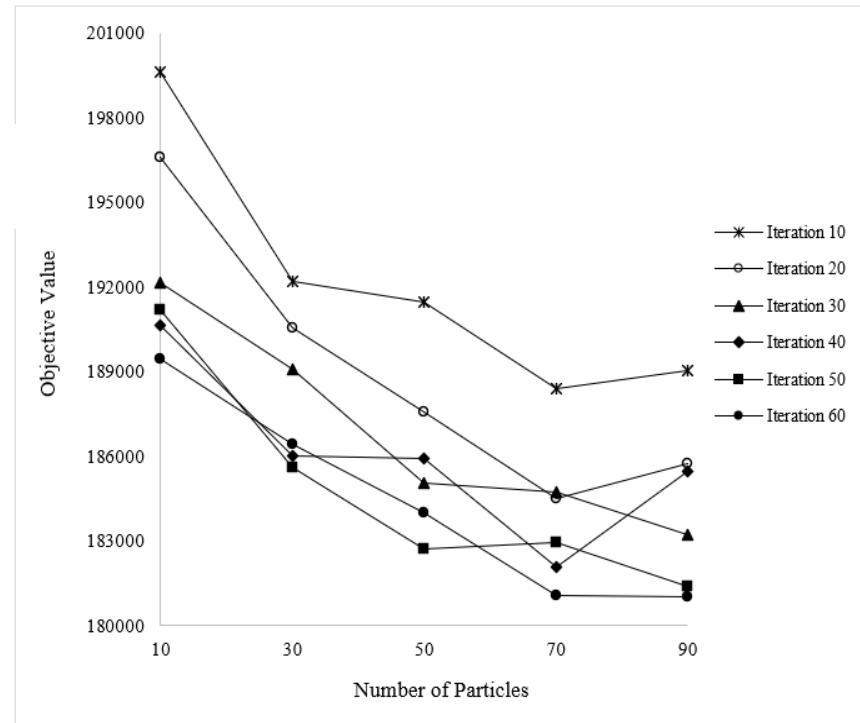
For instance, with 50 iterations: the average solution is obtained as 191,206 with 10 particles, 185,596 with 30 particles, 182,705 with 50 particles, 182,919 with 70 particles, and 181,362 with 90 particles. We can conclude from Figure 5.11 that (a) the PSO algorithm is sensitive to the number of particles. In addition, as increasing the number of particles, the solution quality is also improved. The most significant improvement in the objective value (from 196,599 to 185,739) is observed with 20 iterations as the number of particles increased from 10 to 90. In Figure 5.11 (b), the effect of the number of iterations on the solution quality regarding the objective value can be observed for a specific number of particles. The most significant improvement in the objective value (from 199,631 to 189,424) is observed with 10 particles as the number of iterations increases. We can conclude from Figure 5.11 (b) the PSO algorithm is also sensitive to the number of iterations.

5.4 Performance Evaluation of the MILP Model

In this section the overall performance of the MILP model is evaluated regarding all runs for the problem instances. The MILP model provides the optimum solutions for problem instances 10 scenes/10 actors and 15 scenes/10 actors with all constraints. However, it takes more than 3 hours to solve problem instances larger than 15 scenes/10 actors with the SP-graph types, including blocking time constraints. Similarly, it takes more than 3 hours to solve problem instances larger than 20 scenes/10 actors with the OF-graph type, including blocking time constraints. The MILP model solves most problem instances with the OF-graph types more easily than with the SP-graph types. If we examine all results obtained from the generated problem instances that are smaller than 20 scenes/10 actors with the SP-graph precedence relations, it can be observed that imposing only blocking time constraints in the problem instances increases the complexity of the problems and hence the average computational times. For the problem instances that are larger than 15 scenes/10 actors with only blocking time constraints, the average time required does not decrease.



(a) Effect of Number of Particles



(b) Effect of Number of Iterations

Figure 5.11 Effects of Numbers of Particles and Iterations – Problem Instance: Real Film Script

However, when imposing only time window constraints in the problems, the average computational time decreases. In addition, the lowest average computational time is observed when including all of the constraints regardless of the precedence relations graph types by the CPLEX solver.

The average computational time for the problem instances with the OF-graph precedence relations is less than those with the SP-graph precedence relations when using the CPLEX solver. This means that precedence relations in the OF-graph type enable the problems to be solved more easily by the CPLEX solver.

In the real film script instance, the optimum solution cannot be found due to the size of the problem with 70 scenes and 43 main actors, as mentioned in Section 5.2.2.

5.5 Performance Evaluation of the PSO Algorithm

In this section the overall performance of the PSO algorithm is evaluated regarding all runs for the problem instances. Based on the analyses above, our proposed PSO algorithm provides high-quality near-optimum solutions for the generated problem instances and a real film script instance, as provided in Sections 5.2.1 and 5.2.2, respectively.

Out of 160 generated problem instances, 30 problem instances are solved to optimality; 106 problem instances are solved with a deviation of less than 1.00 %; and only 24 problem instances are solved with a deviation higher than 1.00 % by the PSO algorithm. Furthermore, it is shown that the solution quality can further be increased, and deviations can be reduced to 1.00 % with different numbers of particles and iterations for the problem instances that were originally solved with a limited number of particles and iterations and ended with a deviation higher than 1.00 %. The highest average computational time is observed for the problem instances with both blocking time and time window constraints.

For the problem instances larger than 10 scenes/10 actors, time window constraints cause higher average computational time and percent deviation than blocking time constraints cause, meaning that time window constraint is more challenging for the PSO algorithm.

The problem instances larger than 20 scenes/10 actors with time window constraints and SP-graph types are solved more easily than the ones with blocking time constraints. The opposite is observed for problem instances smaller than 25 scenes/10 actors.

On the other hand, by the PSO algorithm, the average computational time increases when imposing only blocking time constraints in the problem instances with the SP-graph type. However, the opposite is observed for the problem instances with the OF-graph type. Therefore, we can infer that problems involving only blocking time constraints with the OF-graph type are solved in a shorter time than the problem instances with the SP-graph type.

For the real film script instance, most solutions by the PSO algorithm are better than the best integer solution of 190,740 with a 88.87 % optimality gap obtained at the end of 5.6 days by the CPLEX solver. The minimum objective value among all runs is obtained as 165,825 with 70 particles for 20 iterations.

It can be concluded from all runs that the proposed PSO algorithm is sensitive to the numbers of particles and iterations. Solution quality can be improved by increasing the number of particles and iterations for both generated problem instances and a real film script instance. This indicates that the exploration and exploitation characteristics of the proposed PSO algorithm are boosted by increasing the number of particles and iterations. However, computational time increases as we increase the number of particles and iterations. The number of particles has much more impact on the computational time than the number of iterations in all problem instances. Therefore, there is obviously a trade-off between the computational time and the solution quality regarding the objective value.

CHAPTER 6

CONCLUSIONS

In general, scenes that make up a movie are shot in a mixed sequence with a set of actors in a variety of shooting locations all around the world. As a result, actors and shooting crew should be transported from one shooting site to another. Generally, film producers prefer to pay the actors' wages during their accommodation at the shooting sites for both show-up and no-show-up days, rather than paying such an expensive transportation cost that may be incurred due to transferring the actors to their homes to spend their no-show-up days. Therefore, MSSP deals with finding a feasible schedule of scenes to minimize the total cost of movie shooting. In real movie shooting, many factors affect the total cost of a movie shooting. However, in the literature, only the transfer cost of scenes and the actors' wages are considered in the studies for MRSP, TSP, and MSSP.

Motivated by this gap in the literature, we define the MSSP in this thesis, in which most of the realistic factors affecting the total cost are considered. These realistic factors are precedence relations among scenes, time windows of scenes, unavailability (blocking) time periods of actors, and cost factors such as wages of actors, fixed cost incurred due to hiring special shooting sets or sites, cost of using special resources like scenery, costumes and professional digital cameras, and transfer cost of scenes, equipment, actors and shooting crew. Imposing these inherent factors in the problem is one of the major contributions for the MSSP approaches in the literature.

Moreover, a taxonomy is prepared to evaluate the three relevant problems: MRSP, TSP, and MSSP. The previous studies have yet to provide a taxonomy as detailed as the one presented in this thesis. Graphs for the chronological hierarchy are also formed for these problem types.

Moreover, the MILP model is developed according to the MSSP environment and related assumptions as defined in Chapter 3. The objective function includes actors' wages during their available time, transfer cost of scenes, equipment, actors and shooting crew, fixed cost incurred due to hiring special shooting sets or sites and cost of utilizing professional equipment. The constraints satisfy the precedence relations among scenes, earliest starting and latest completion times of scenes, and blocking time periods of actors in which they go on-leave for a pre-specified duration.

In addition, a discrete PSO algorithm is developed to solve the large-scale problems that cannot be solved to optimality by the CPLEX solver. The performances of the MILP model and proposed PSO algorithm are evaluated based on several problem instances generated and a real film script instance, as well. By the PSO algorithm, most problem instances are solved to optimality or close to optimality with a deviation of less than 1.00%. Moreover, it is shown that the solution quality can be increased, and deviations are reduced to 1.00% using several numbers of particles and iterations for the problem instances with a deviation higher than 1.00%.

For a future research direction, the MILP can be developed so that the film producer can also decide whether to keep the actors continuously at shooting sets or send them back to their homes by comparing the costs of these options.

In a real movie shooting, due to some uncertain factors such as shooting crew absence, equipment failure, performance effects, etc., the duration of the scenes may not be deterministic and known in advance. Therefore, the uncertainty in the scene durations can be considered in future studies.

Moreover, valid inequalities can be added to the MILP model as an attempt to shrink the solution region. In this way, the search space is reduced, and as a result, long run times can be decreased, and optimal solutions may be found in a shorter time.

In our MSSP, we assume that only one scene can be shot at a time since there is only one shooting crew. As an extension to our study, parallel shootings, as in the case of parallel machine scheduling problem, can be allowed with more than a shooting crew by aiming to reduce the total movie shooting completion and accordingly cost of shooting. Moreover, different metaheuristics, such as TS and ACO algorithms, can be developed to compare our PSO algorithm against as another future research issue.

While our proposed MILP model and PSO algorithm are developed specifically based on the MSSP, we expect these methods to be useful for some other related combinatorial problems, like the project scheduling problem, that share some common characteristics with the MSSP. For instance, scenes with precedence relations and time windows, actors with blocking times, and shooting schedules in the MSSP may be viewed as activities, resources, and project schedules in project scheduling problems.

REFERENCES

- Ab Rashid, M. F. F., Hutabarat, W., & Tiwari, A. (2012). Development of a tuneable test problem generator for assembly sequence planning and assembly line balancing. In *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture*, 226(11), 1900-1913.
- Adelson, R. M., Norman, J. M., & Laporte, G. (1976). A dynamic programming formulation with diverse applications. *Operational Research Quarterly*, 119-121.
- Anghinolfi, D., Montemanni, R., Paolucci, M., & Gambardella, L. M. (2011). A hybrid particle swarm optimization approach for the sequential ordering problem. *Computers & Operations Research*, 38(7), 1076-1085.
- Bomsdorf, F., & Derigs, U. (2008). A model, heuristic procedure and decision support system for solving the movie shoot scheduling problem. *OR Spectrum*, 30(4), 751-772.
- Cheng, T. C. E., Diamond, J. E., & Lin, B. M. (1993). Optimal scheduling in film production to minimize talent hold cost. *Journal of Optimization Theory and Applications*, 79(3), 479-492.
- Cheng, T. C. E., Lin, B. M. T., & Huang, H. L. (2017). Talent hold cost minimization in film production. *Journal of Industrial & Management Optimization*, 13(1), 223-235.
- Davari, M., Demeulemeester, E., Leus, R., & Talla Nobibon, F. (2016). Exact algorithms for single-machine scheduling with time windows and precedence constraints. *Journal of Scheduling*, 19(3), 309-334.
- Ding, H., Fan, Y., & Zhong, W. (2018). Robust optimization of performance scheduling problem under accepting strategy. *Open Journal of Optimization*, 7(4), 65-78.

- Eberhart, R. C., Shi, Y., & Kennedy, J. (2001). *Swarm intelligence*. Elsevier.
- Fink, A., & Voß, S. (1999). Applications of modern heuristic search methods to pattern sequencing problems. *Computers & Operations Research*, 26(1), 17-34.
- Garcia de la Banda, M., Stuckey, P. J., & Chu, G. (2011). Solving talent scheduling with dynamic programming. *INFORMS Journal on Computing*, 23(1), 120-137.
- Glover, F., & Laguna, M. (1998). Tabu search. In *Handbook of Combinatorial Optimization* (pp. 2093-2229). Springer, Boston, MA.
- Gregory, P., Miller, A., & Prosser, P. (2004, August). Solving the rehearsal problem with planning and with model checking. In *Proceedings of the Workshop on Modelling and Solving Problems with Constraints. Held in conjunction with the 16th European Conference on Artificial Intelligence (ECAI 2004)* (pp. 157-171).
- He, Q., & Wang, L. (2007). A hybrid particle swarm optimization with a feasibility-based rule for constrained optimization. *Applied Mathematics and Computation*, 186(2), 1407-1422.
- Hill, E., & Velednitsky, M. (2021). Rehearsal scheduling: Developing an optimization solution with practitioner input. In *Proceedings of the 13th International Conference on the Practice and Theory of Automated Timetabling-PATAT* (Vol. 1).
- Jarboui, B., Damak, N., Siarry, P., & Rebai, A. (2008). A combinatorial particle swarm optimization for solving multi-mode resource-constrained project scheduling problems. *Applied Mathematics and Computation*, 195(1), 299-308.
- Jarumaneeroj, P., & Sakulsom, N. (2021). An adaptive large neighborhood search for the multiple-day music rehearsal problems. *Computers & Industrial Engineering*, 157, 107279.

- Jayanthi, S. E., & Anusuya, S. (2017). Minimization of total weighted earliness and tardiness using PSO for one machine scheduling. *International Journal of Pure and Applied Mathematical Sciences*, 10(1), 35-44.
- Jia, Q., & Seo, Y. (2013). An improved particle swarm optimization for the resource-constrained project scheduling problem. *The International Journal of Advanced Manufacturing Technology*, 67(9), 2627-2638.
- Joy, J., Rajeev, S., & Narayanan, V. (2016). Particle swarm optimization for resource constrained-project scheduling problem with varying resource levels. *Procedia Technology*, 25, 948-954.
- Kennedy, J., & Eberhart, R. (1995, November). Particle swarm optimization. In *Proceedings of the ICNN'95-International Conference on Neural Networks* (Vol. 4, pp. 1942-1948). IEEE.
- Kennedy, J., & Eberhart, R. C. (1997, October). A discrete binary version of the particle swarm algorithm. In *1997 IEEE International Conference on Systems, Man, and Cybernetics. Computational Cybernetics and Simulation* (Vol. 5, pp. 4104-4108). IEEE.
- Khalilzadeh, M., Kianfar, F., Shirzadeh Chaleshtari, A., Shadrokh, S., & Ranjbar, M. (2012). A modified PSO algorithm for minimizing the total costs of resources in MRCPSP. *Mathematical Problems in Engineering*, 2012.
- Kochetov, Y. (2011, September). Iterative local search methods for the talent scheduling problem. In *Proceedings of the 1st International Symposium and 10th Balkan Conference on Operational Research, September* (Vol. 22, pp. 282-288).
- Koulinas, G., Kotsikas, L., & Anagnostopoulos, K. (2014). A particle swarm optimization based hyper-heuristic algorithm for the classic resource constrained project scheduling problem. *Information Sciences*, 277, 680-693.

- Liao, C. J., Tseng, C. T., & Luarn, P. (2007). A discrete version of particle swarm optimization for flowshop scheduling problems. *Computers & Operations Research*, 34(10), 3099-3111.
- Liu, B. (2016, April). Scheduling of rehearsals with mixing piece length. In *Proceedings of the International Conference on Education, Management and Computing Technology (ICEMCT-16)* (pp. 1003-1006). Atlantis Press.
- Liu, Y., Sun, Q., Zhang, X., & Wu, Y. (2019). Research on the scheduling problem of movie scenes. *Discrete Dynamics in Nature and Society*, 2019.
- Long, X., & Zhao, J. (2020). Scheduling problem of movie scenes based on three meta-heuristic algorithms. *IEEE Access*, 8, 59091-59099.
- Low, C., Hsu, C. J., & Su, C. T. (2010). A modified particle swarm optimization algorithm for a single-machine scheduling problem with periodic maintenance. *Expert Systems with Applications*, 37(9), 6429-6434.
- Naderi, B., Gohari, S., & Yazdani, M. (2014). Hybrid flexible flowshop problems: Models and solution methods. *Applied Mathematical Modelling*, 38(24), 5767-5780.
- Nordström, A. L., & Tufekci, S. (1994). A genetic algorithm for the talent scheduling problem. *Computers & Operations Research*, 21(8), 927-940.
- Pakprajum, C., & Jarumaneeroj, P. (2017, April). An integrated MILP for music rehearsal problems with waiting time. In *Proceedings of the 4th International Conference on Industrial Engineering and Applications (ICIEA)* (pp. 214-218). IEEE.
- Petropoulos, D. I., & Nearchou, A. C. (2011). A particle swarm optimization algorithm for balancing assembly lines. *Assembly Automation*, 31(2), 118-129.
- Qin, H., Zhang, Z., Lim, A., & Liang, X. (2014). An enhanced branch-and-bound algorithm for the talent scheduling problem. *ArXiv Preprint ArXiv:1401.5869*.

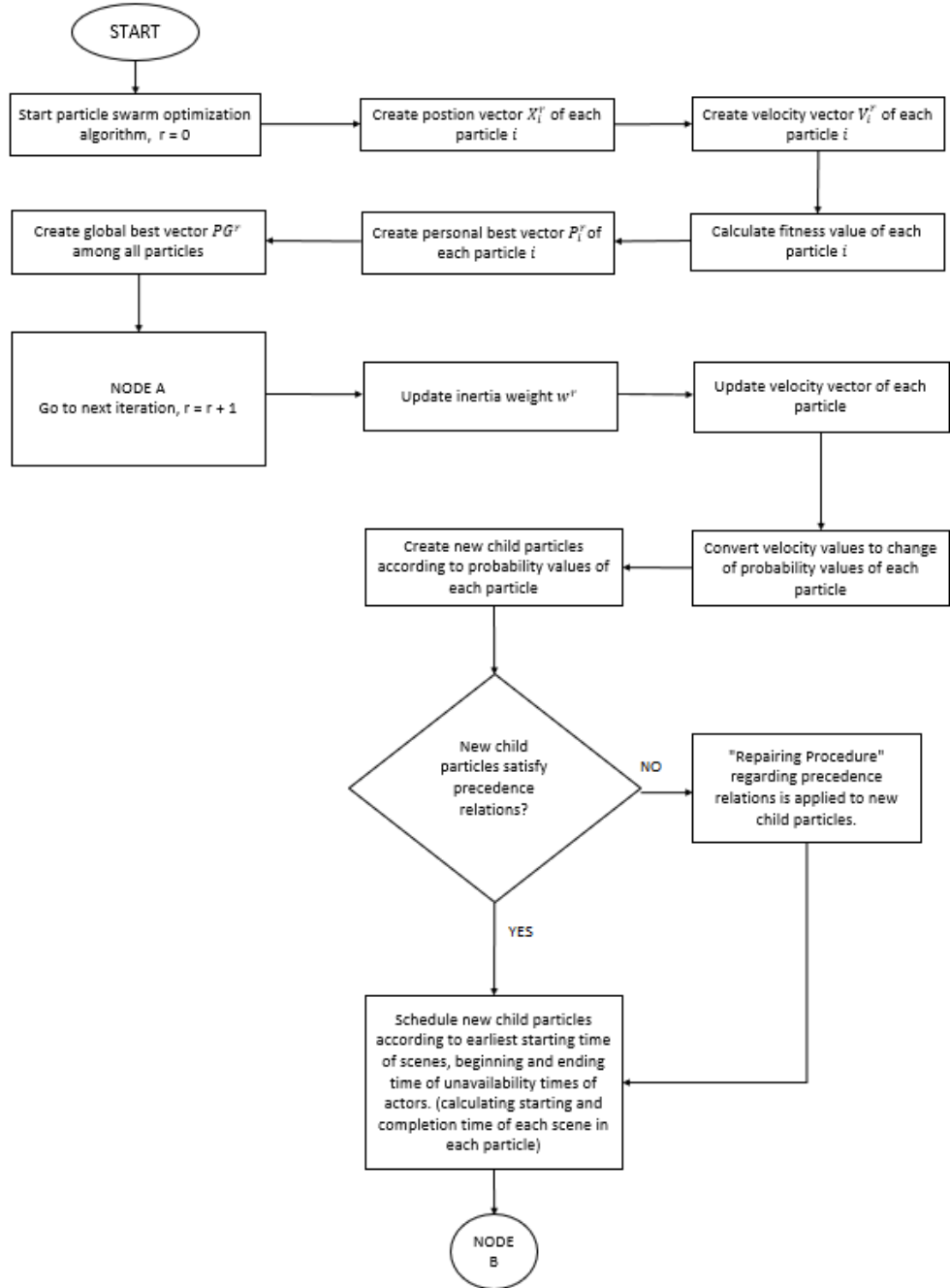
- Qin, H., Zhang, Z., Lim, A., & Liang, X. (2016). An enhanced branch-and-bound algorithm for the talent scheduling problem. *European Journal of Operational Research*, 250(2), 412-426.
- Piotrowski, A. P., Napiorkowski, J. J., & Piotrowska, A. E. (2020). Population size in particle swarm optimization. *Swarm and Evolutionary Computation*, 58, 100718.
- Prot, D., & Bellenguez-Morineau, O. (2018). A survey on how the structure of precedence constraints may change the complexity class of scheduling problems. *Journal of Scheduling*, 21(1), 3-16.
- Ranjbar, M., & Kazemi, A. (2018). A generalized variable neighborhood search algorithm for the talent scheduling problem. *Computers & Industrial Engineering*, 126, 673-680.
- Sakulsom, N., & Tharmmaphornphilas, W. (2011). A multi-objective music rehearsal scheduling problem. In *Proceedings of 12th Asia Pacific Industrial Engineering & Management Systems Conference (APIEMS 2011)*.
- Sakulsom, N., & Tharmmaphornphilas, W. (2014). Scheduling a music rehearsal problem with unequal music piece length. *Computers & Industrial Engineering*, 70, 20-30.
- Shi, Y., & Eberhart, R. C. (1998, March). Parameter selection in particle swarm optimization. In *Proceedings of the International Conference on Evolutionary Programming* (pp. 591-600). Springer, Berlin, Heidelberg.
- Smith, B. M. (2003). Constraint programming in practice: Scheduling a rehearsal. *Research Report APES-67-2003*, APES group.
- Smith, B. M. (2005, October). Caching search states in permutation problems. In *Proceedings of the International Conference on Principles and Practice of Constraint Programming* (pp. 637-651). Springer, Berlin, Heidelberg.

- Şahin, Ö. G. (2007). Masal. Istanbul Bilgi University, School of Film and Television.
- Taşgetiren, M. F., Sevkli, M., Liang, Y. C., & Gençyılmaz, G. (2004, June). Particle swarm optimization algorithm for single machine total weighted tardiness problem. In *Proceedings of the 2004 Congress on Evolutionary Computation* (IEEE Cat. No. 04TH8753) (Vol. 2, pp. 1412-1419). IEEE.
- Tchomte, S. K., & Gourgand, M. (2009). Particle swarm optimization: A study of particle displacement for solving continuous and combinatorial optimization problems. *International Journal of Production Economics*, 121(1), 57-67.
- Otto, A., Otto, C., & Scholl, A. (2011). *SALBPGen-A systematic data generator for (simple) assembly line balancing* (No. 05/2011). Friedrich-Schiller-University Jena, School of Economics and Business Administration.
- Wang, S. Y., Chuang, Y. T., & Lin, B. M. (2016). Minimizing talent cost and operating cost in film production. *Journal of Industrial and Production Engineering*, 33(1), 17-31.
- Wang, X. J. (2015, November). Optimization of production cost of film and its solution method. In *Proceedings of the International Conference on Engineering Management, Engineering Education and Information Technology* (pp. 537-542). Atlantis Press.
- Yan, P., Wang, J. B., & Zhao, L. Q. (2019). Single-machine bi-criterion scheduling with release times and exponentially time-dependent learning effects. *Journal of Industrial & Management Optimization*, 15(3), 1117.
- Yang, X. S., Deb, S., & Fong, S. (2014). Metaheuristic algorithms: Optimal balance of intensification and diversification. *Applied Mathematics & Information Sciences*, 8(3), 977.

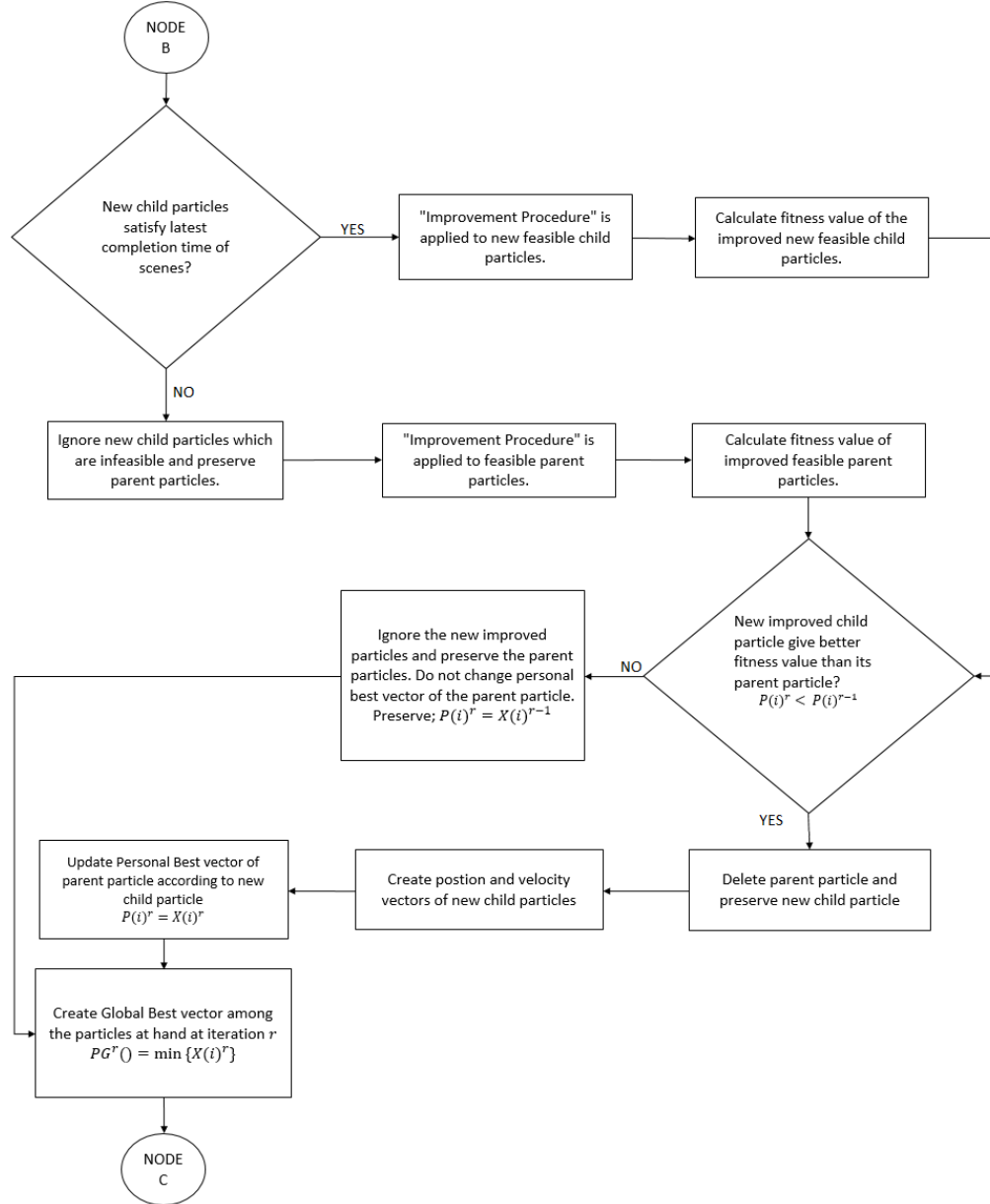
Zhang, H., Li, H., & Tam, C. M. (2006). Particle swarm optimization for resource-constrained project scheduling. *International Journal of Project Management*, 24(1), 83-92.

APPENDICES

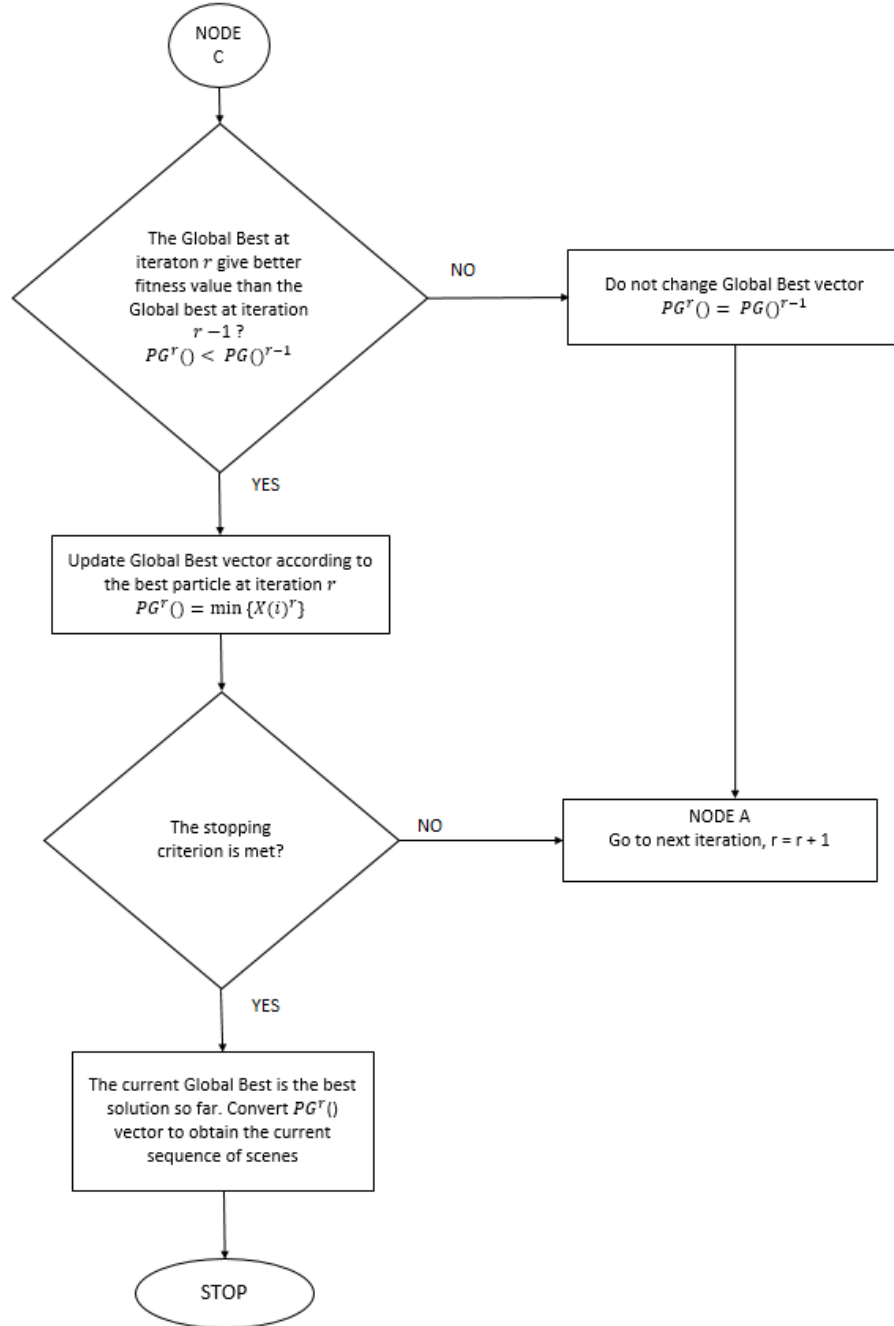
A. FLOWCHART OF THE PROPOSED PSO ALGORITHM



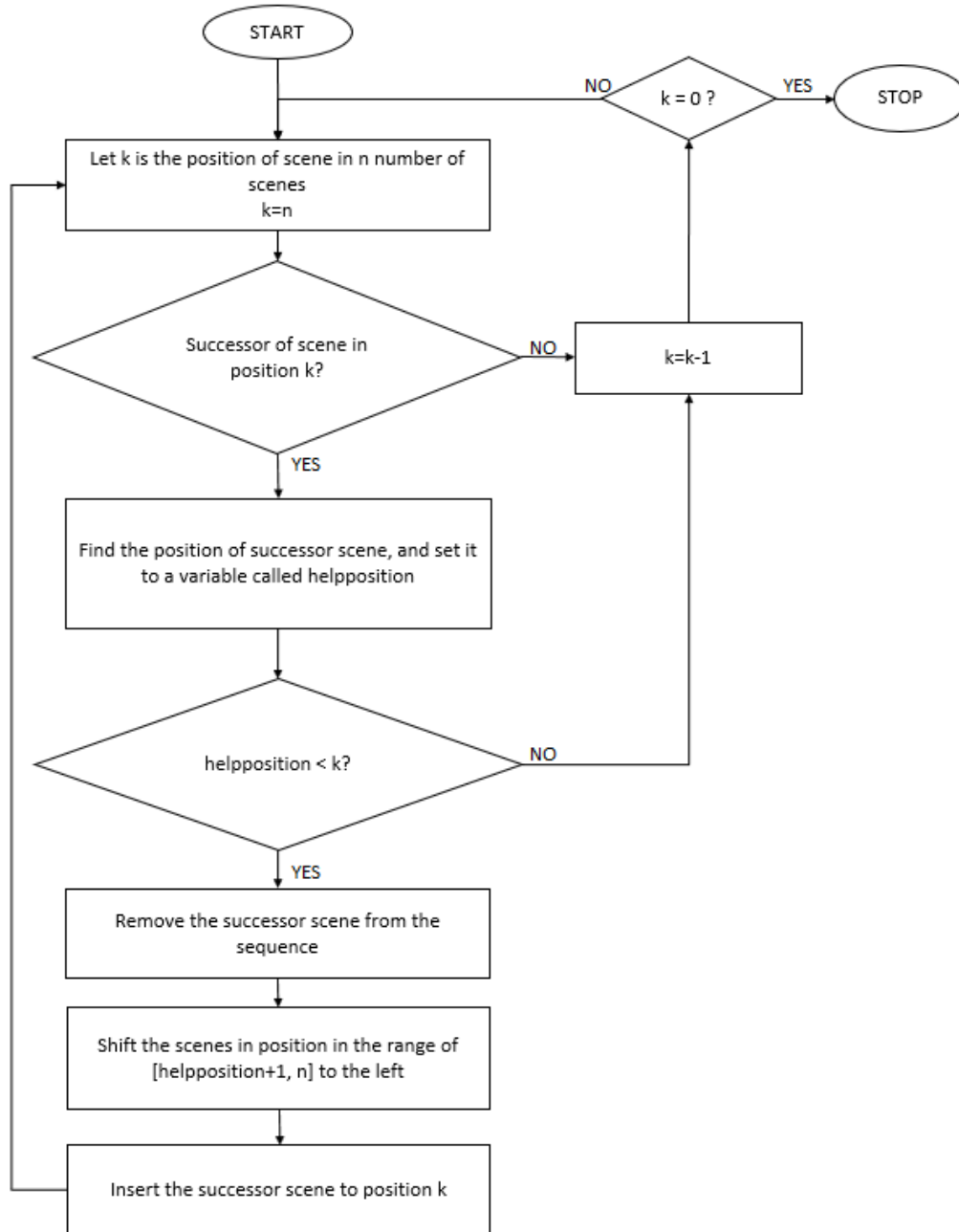
A. FLOWCHART OF THE PROPOSED PSO ALGORITHM-continued



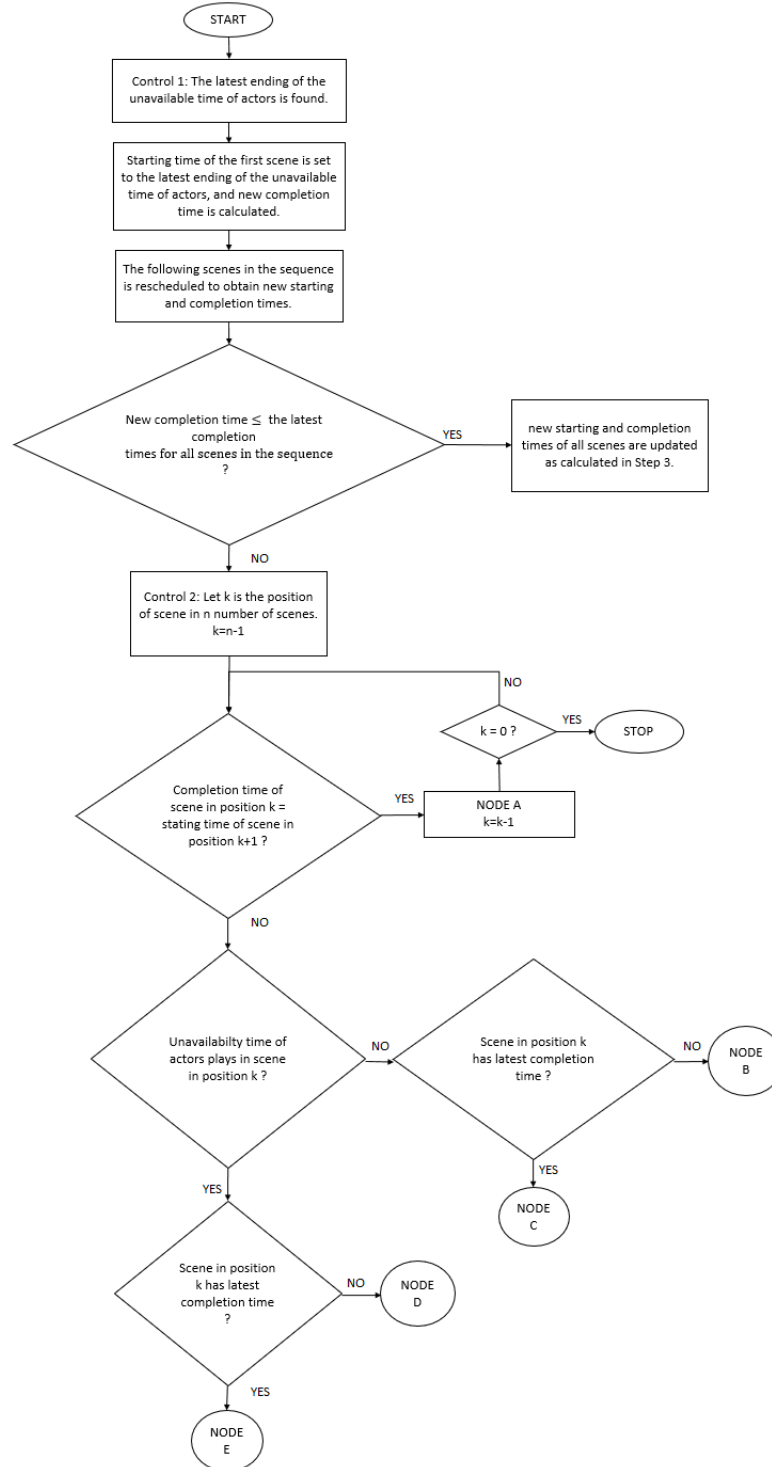
A. FLOWCHART OF THE PROPOSED PSO ALGORITHM- continued



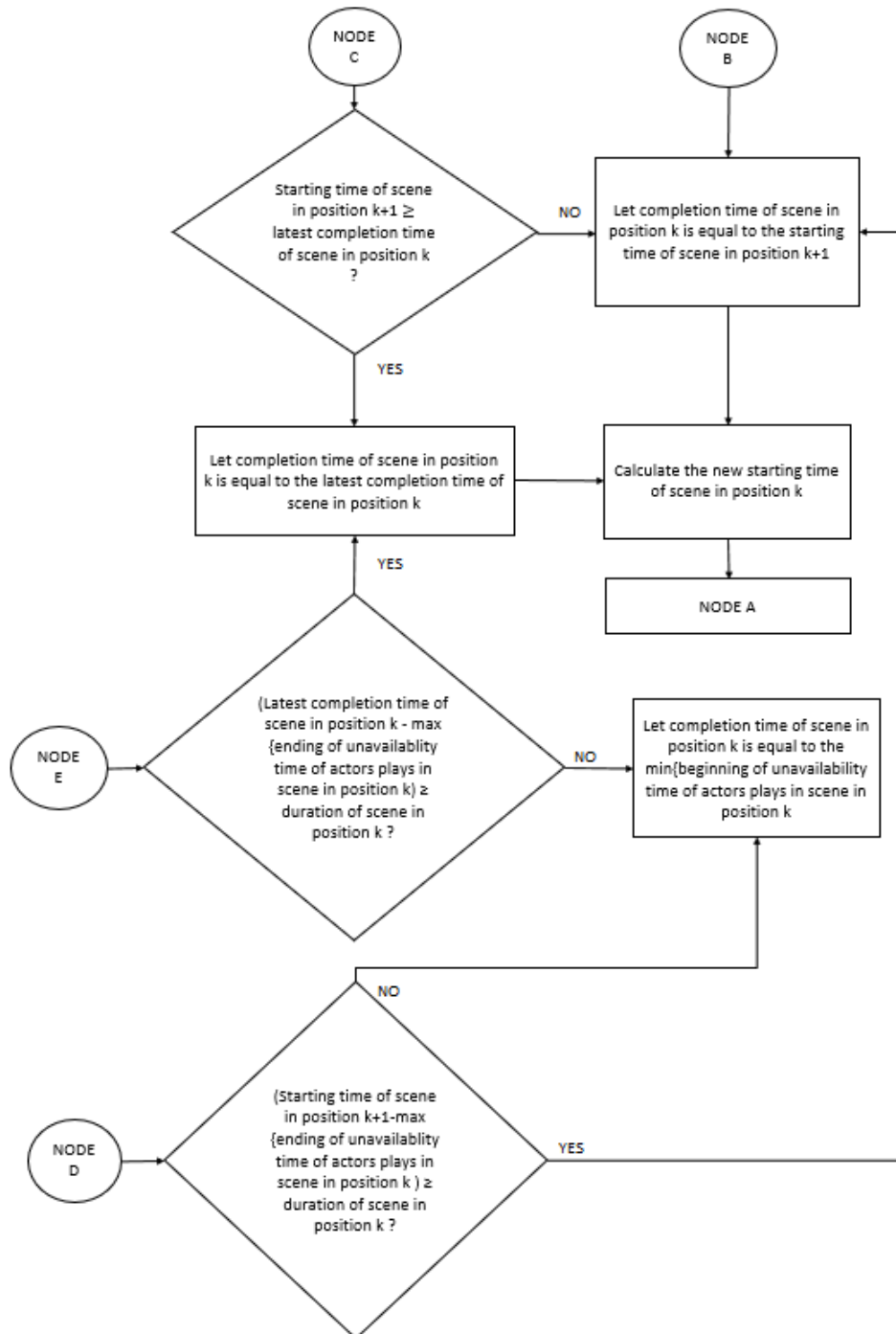
B. FLOWCHART OF THE REPAIRING PROCEDURE



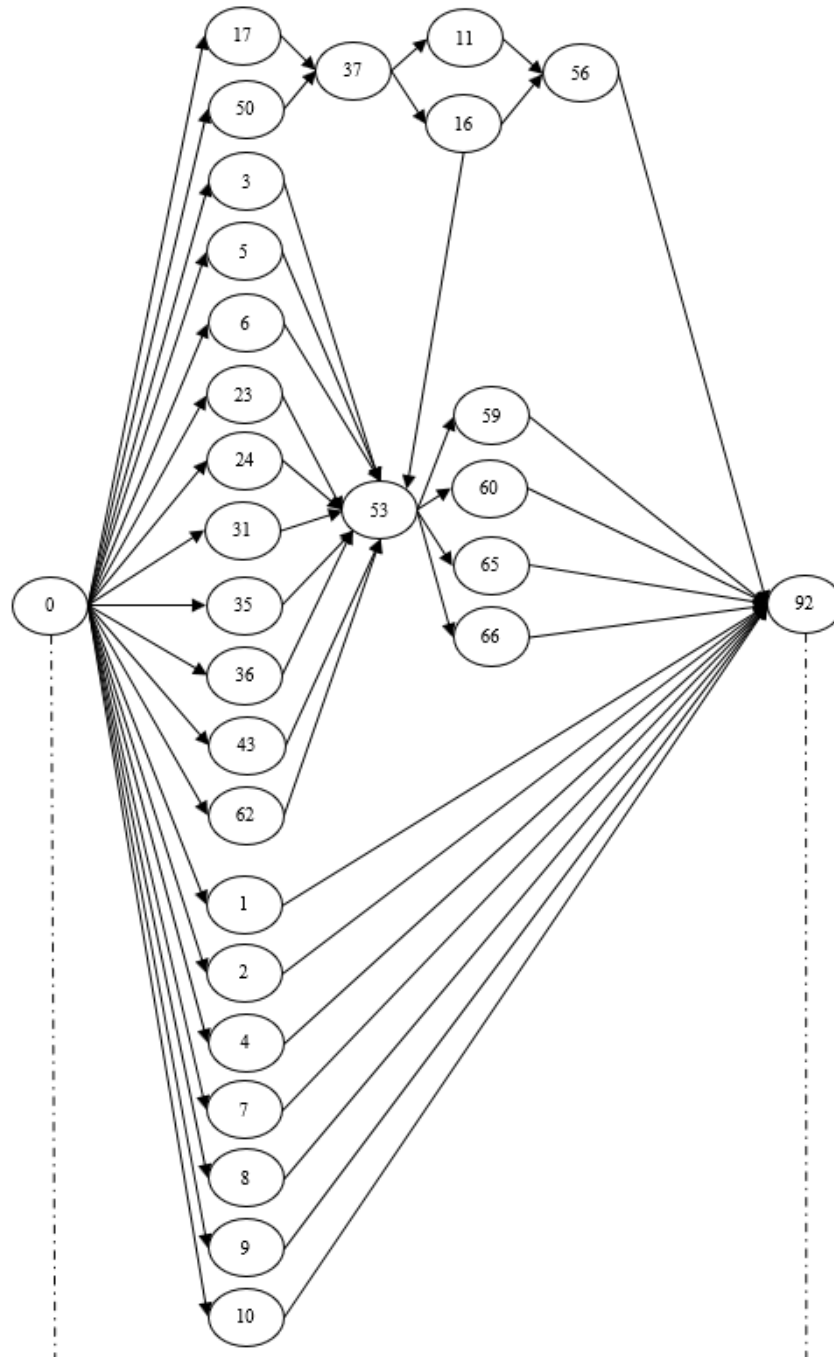
C. FLOWCHART OF THE IMPROVEMENT PROCEDURE



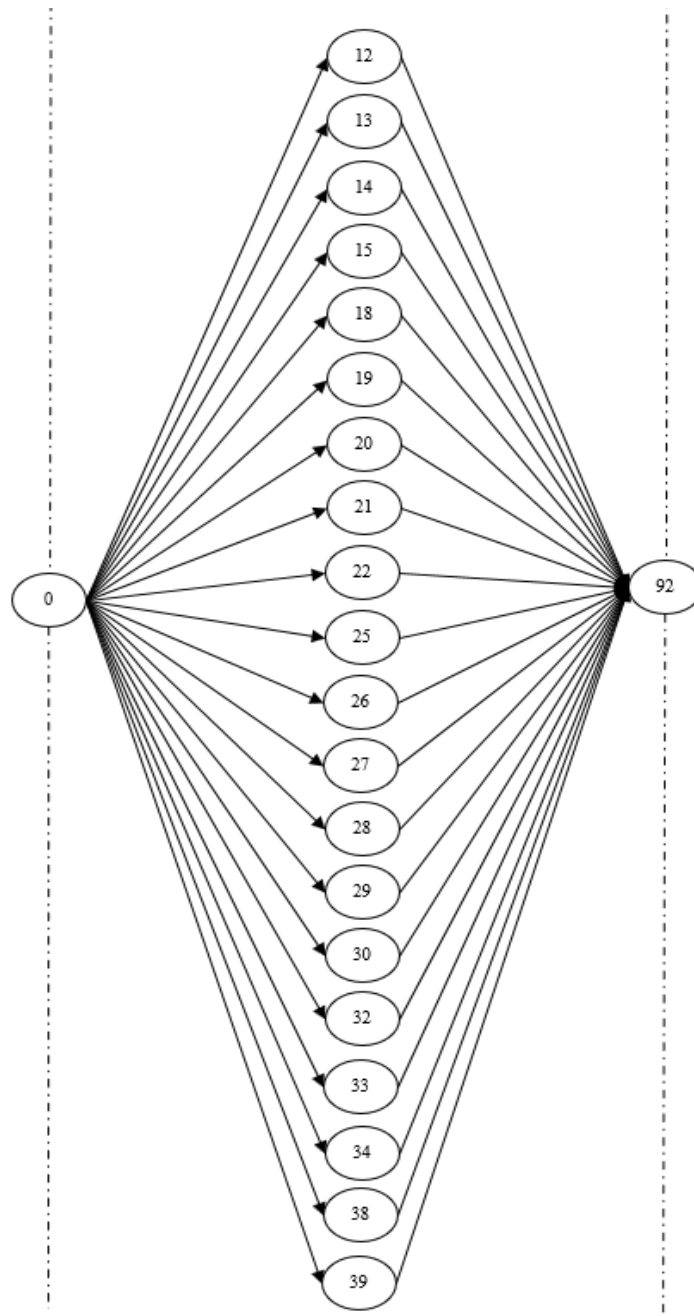
C. FLOWCHART OF THE IMPROVEMENT PROCEDURE- continued



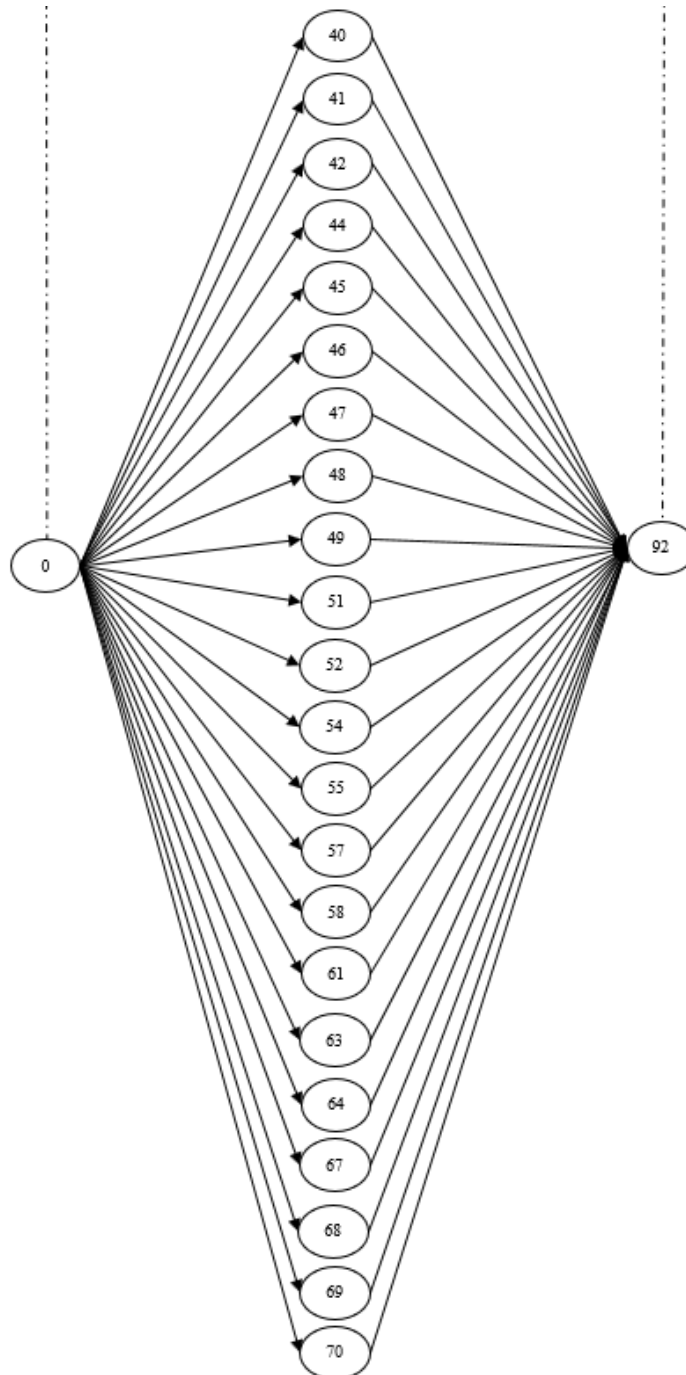
D. PRECEDENCE RELATIONS AMONG THE SCENES IN THE REAL FILM SCRIPT



**D. PRECEDENCE RELATIONS AMONG THE SCENES IN THE REAL
FILM SCRIPT- continued**



**D. PRECEDENCE RELATIONS AMONG THE SCENES IN THE REAL
FILM SCRIPT- continued**



**E. SCENE NUMBERS IN THE PRECEDENCE DIAGRAM AND IN ŞAHİN
(2007)**

SCENE NUMBER IN ŞAHİN (2007)	SCENE NUMBER IN THE PRECEDENCE RELATIONS
SAHNE 1-2-3	1
SAHNE 4	2
SAHNE 5	3
SAHNE 6	4
SAHNE 7-27	5
SAHNE 8	6
SAHNE 9-10	7
SAHNE 11	8
SAHNE 12	9
SAHNE 13	10
SAHNE 14	11
JENERIK1 (PAGE 19)	12
SAHNE 16	13
SAHNE 17	14
SAHNE 18	15
SAHNE 19-20	16
SAHNE 21	17
SAHNE 22	18
SAHNE 23	19
SAHNE 24	20
SAHNE 25	21
SAHNE 26	22
SAHNE 28	23
SAHNE 29-30	24
JENERIK2 (PAGE 106)	25
SAHNE 32	26
SAHNE 33	27
SAHNE 34	28
SAHNE 35	29
SAHNE 36	30
SAHNE 37	31

**E. SCENE NUMBERS IN THE PRECEDENCE DIAGRAM AND IN ŞAHİN
(2007)- continued**

SCENE NUMBER IN ŞAHİN (2007)	SCENE NUMBER IN THE PRECEDENCE RELATIONS
SAHNE 38-39	32
SAHNE 40-41	33
SAHNE 42	34
SAHNE 43	35
SAHNE 44	36
DUMMY FOR DAY-NIGHT	37
SAHNE 46	38
SAHNE 47	39
SAHNE 48	40
SAHNE 49	41
SAHNE 50	42
SAHNE 51	43
SAHNE 52	44
SAHNE 53	45
SAHNE 54	46
SAHNE 55	47
SAHNE 56	48
SAHNE 57	49
SAHNE 58-59	50
SAHNE 60	51
SAHNE 61-63	52
DUMMY FOR GÜROL	53
SAHNE 64	54
SAHNE 65	55
SAHNE 66	56
SAHNE 67-68	57
SAHNE 69	58
SAHNE 70	59
SAHNE 71-72	60
SAHNE 73	61
SAHNE 74	62
SAHNE 75	63
SAHNE 76	64
SAHNE 78	65
SAHNE 79-80-81	66
SAHNE 82	67
SAHNE 83	68
SAHNE 84-85	69
SAHNE 86	70

F. SCENE-ACTOR MATRIX IN THE REAL FILM SCRIPT

[illegible]